Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons → **today's lab**
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

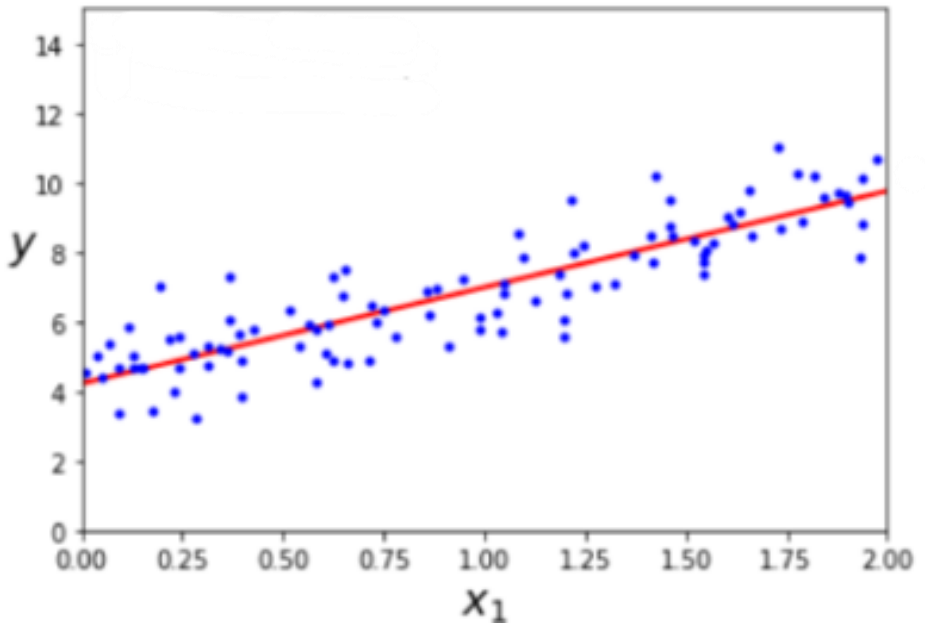
- Linear regression in PyTorch
- The FashionMNIST dataset
- Softmax regression in PyTorch
- Multilayer perceptrons
- Weight decay
- Dropout
- Deep learning computation

1) Linear regression in PyTorch

➢ the linear regression model is expressed as:

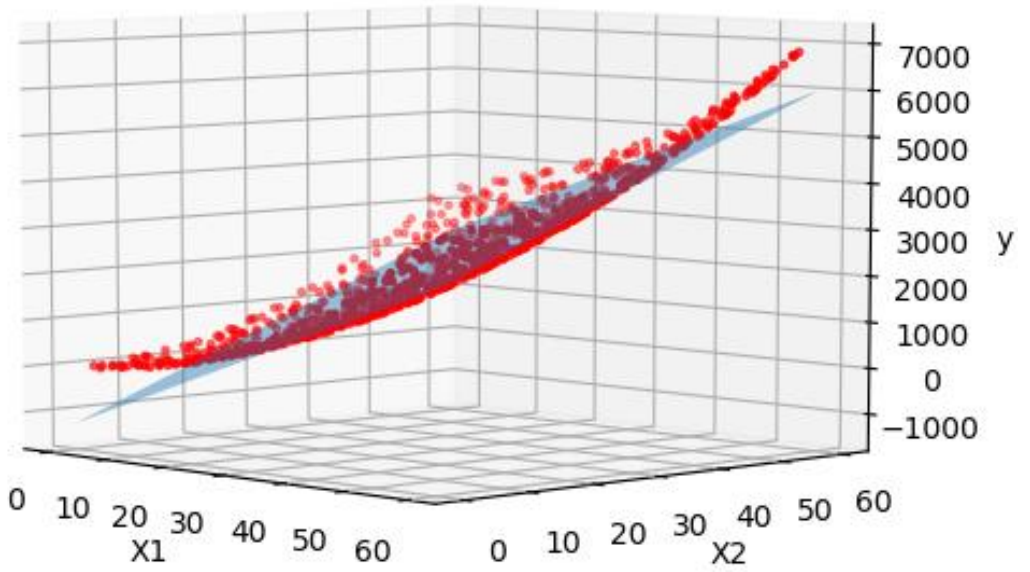
$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d.$$

- $\hat{y}$: predicted value
- w_0: bias
- w_1: feature weight
- x_1: feature
- w_d: feature weight
- x_d: feature



d=2

$\hat{y} = w_0 + w_1 x_1$ → equation of a line



d=3

$\hat{y} = w_0 + w_1 x_1 + w_2 x_2$ → equation of a plane

NOTE: We cannot plot more than 3 dimensions ! ☺ (actually we can but it's more tricky)

```python
def synthetic_data(w, b, num_examples):
    """Generate y = Xw + b + noise."""
    X = torch.normal(0, 1, (num_examples, len(w)))
    y = X @ w + b
    y += torch.normal(0, 0.01, y.shape)
    return X, y.reshape(-1, 1)
true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)
```

Basically here we created: $y = 4.2 + 2x_1 - 3.4x_2 + \epsilon$

features

labels

w_0 (or b)

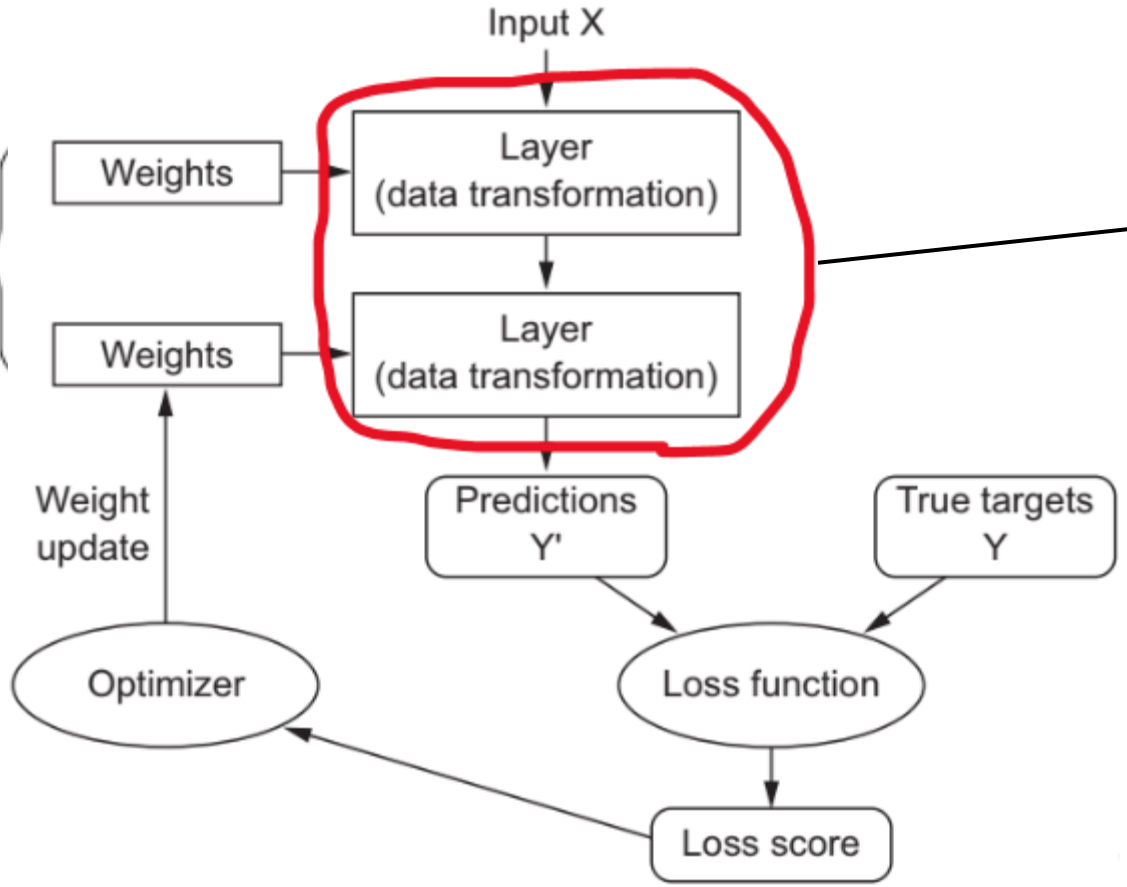
w_1

w_2

noise

Now we want to apply Linear Regression to find these values: w_0 (or b – bias), w_1, w_2 !

Goal: finding the right values for these weights



DL model

Steps

- construct the dataset, shuffle the dataset and access it in mini-batches:

```
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays)
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train)

batch_size = 10
data_iter = load_array((features, labels), batch_size)
```

- define the linear regression model:

```
net = nn.Sequential(nn.Linear(2, 1))
```

we will discuss later about it

- 2 because we have 2 inputs (x_1, x_2)
- 1 because we have 1 output (y)

- initialize the parameters:

```
net[0].weight.data.normal_(0, 0.01)
net[0].bias.data.fill_(0)
```

- specify the loss function to be used:

```
loss = nn.MSELoss()
```

Steps

- specify the optimizer to be used:

```
optimizer = torch.optim.SGD(net.parameters(), lr=0.03)
```

- generate predictions (*net(X)*) and compute the loss (**forward propagation**)
- calculate gradients by running the **backpropagation** using *backward()* on the loss
- update the model parameters using *optimizer.step()*: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$ 
- compute the loss after each epoch and print it to the monitor:

```
num_epochs = 3
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y)
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
    l = loss(net(features), labels)
    print(f'Epoch {epoch + 1}, Loss {l:f}')
```

```
Epoch 1, Loss 0.000353
Epoch 2, Loss 0.000103
Epoch 3, Loss 0.000101
```

NOTE:

- this operation is essential to avoid accumulating gradients from previous operations
- in PyTorch, the gradients are accumulated (summed up) by default in the *.grad* attribute of tensors whenever *backward()* is called
- this is helpful for cases where you want to accumulate gradients across multiple operations, but it can lead to incorrect or unintended results if you're not explicitly aiming for that

NOTE:

Result:

```
w = net[0].weight.data
b = net[0].bias.data
print(b, w)
```

```
tensor([4.2011]) tensor([[ 1.9999, -3.3985]])
```

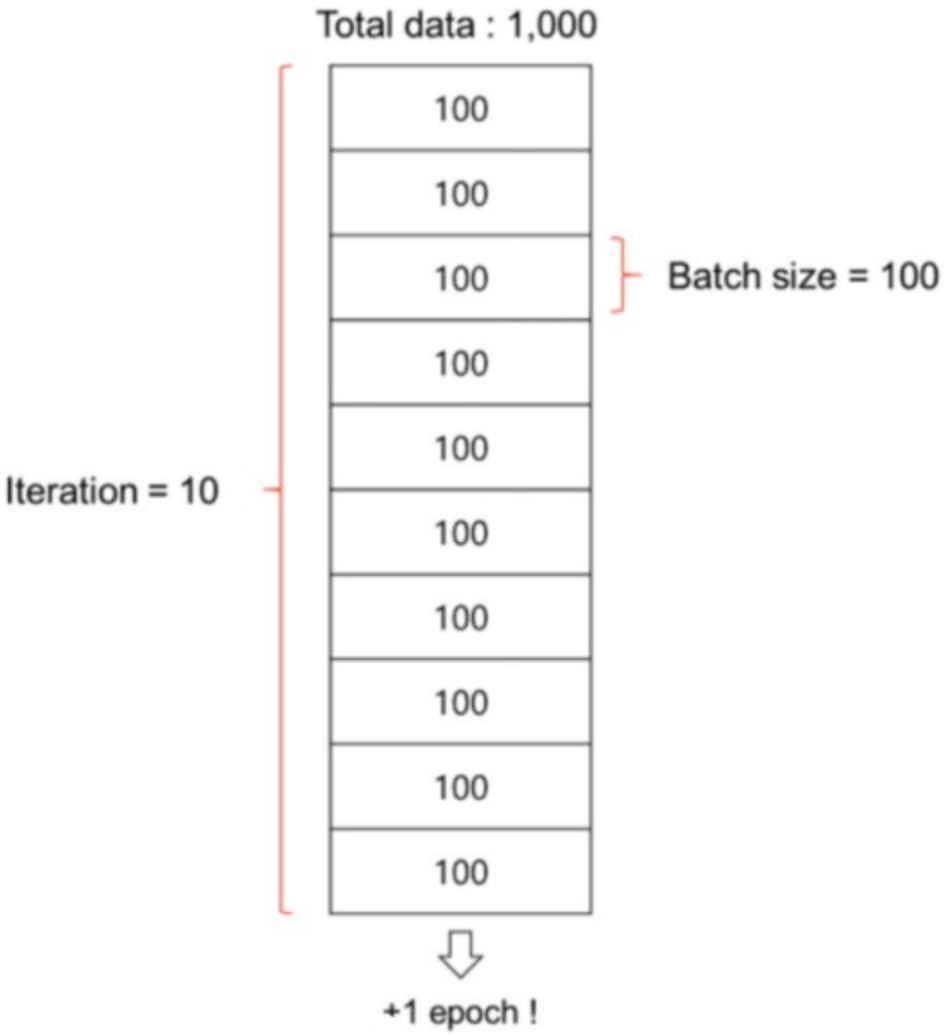
Our true w and b: $y = 4.2 + 2x_1 - 3.4x_2 + \epsilon$

w_0 (or b) w_1 w_2 noise

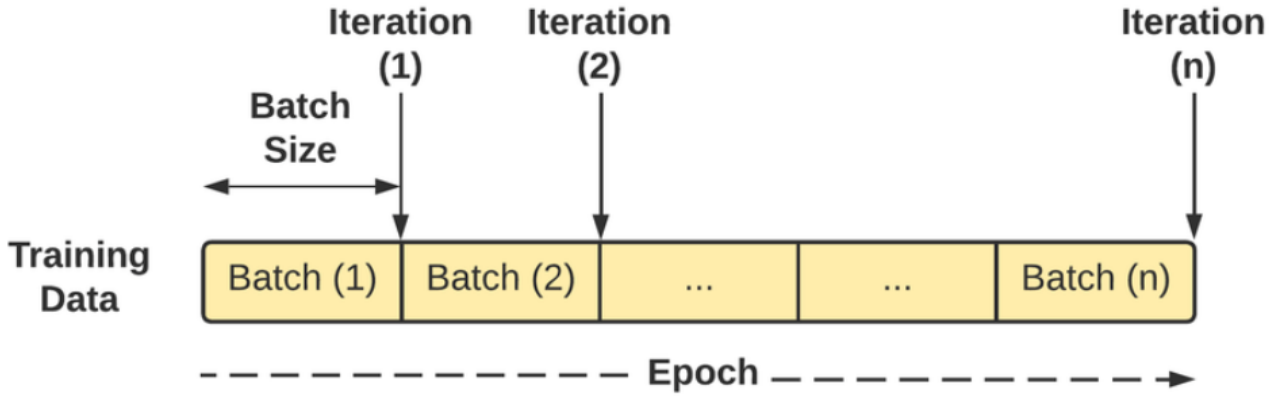
w



Kind reminder



- a **batch** is a subset of the training dataset used to calculate the loss and update the model parameters during one forward and backward pass
- an **iteration** is one step of training where a single batch is processed
- an **epoch** is a complete pass through the entire training dataset



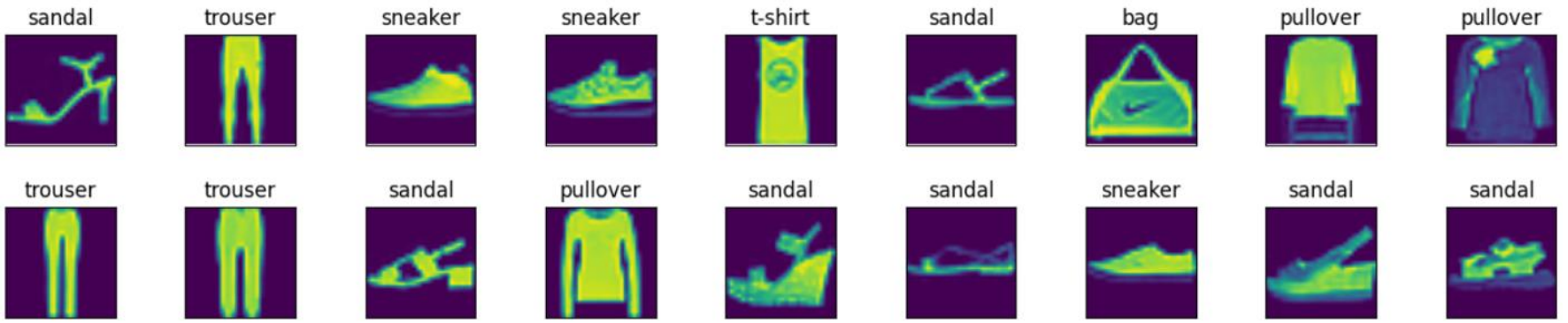
2) The FashionMNIST dataset

- **description:** a dataset of 70,000 grayscale images of clothing items, with 10 classes (e.g., T-shirt, trousers, etc.):
    - training set: 60,000 images
    - test set: 10,000 images

- **image size:** each image is 28×28 pixels

- **classes:**

1. T-shirt/top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle boot



```python
def load_data_fashion_mnist(batch_size, resize=None):
    """Download the Fashion-MNIST dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(root="../data", train=False, transform=trans, download=True)
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [50000, 10000], generator=torch.Generator().manual_seed(42))

    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))
```

- converts the image data from PIL type to 32-bit floating point tensors
- divides all numbers by 255, so that all pixel values are between 0 and 1

```python
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

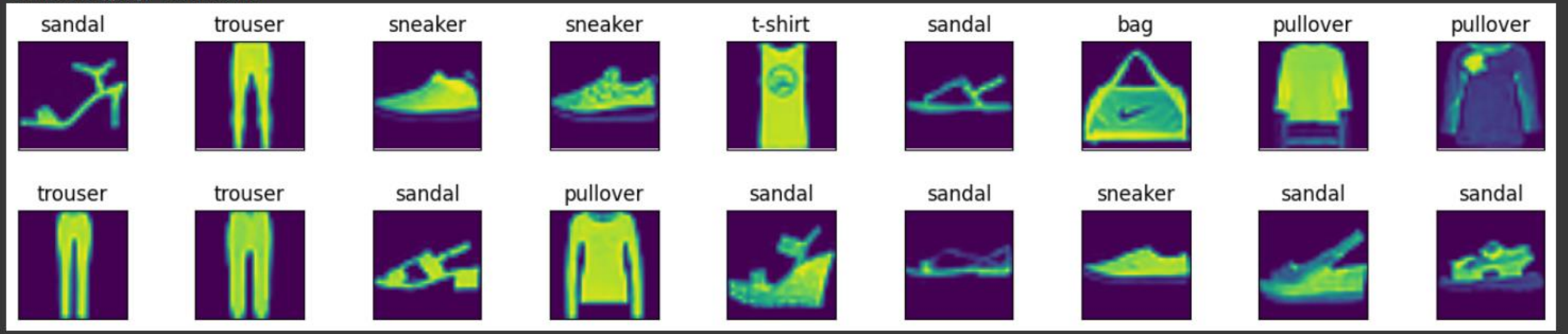
```python
def get_fashion_mnist_labels(labels):
    """Return text labels for the Fashion-MNIST dataset."""
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [text_labels[int(i)] for i in labels]
```

NOTE: functions already done in lab, just use them as they are ☺

Here are the images and their corresponding labels (in text) for the first few (18) examples in the training dataset:

```
train_iter, val_iter, test_iter = load_data_fashion_mnist(18, resize=64)
for X, y in train_iter:
    print(X.shape, X.dtype)
    print(y.shape, y.dtype)
    show_images(X.reshape(18, 64, 64), 2, 9, titles=get_fashion_mnist_labels(y))
    break
```

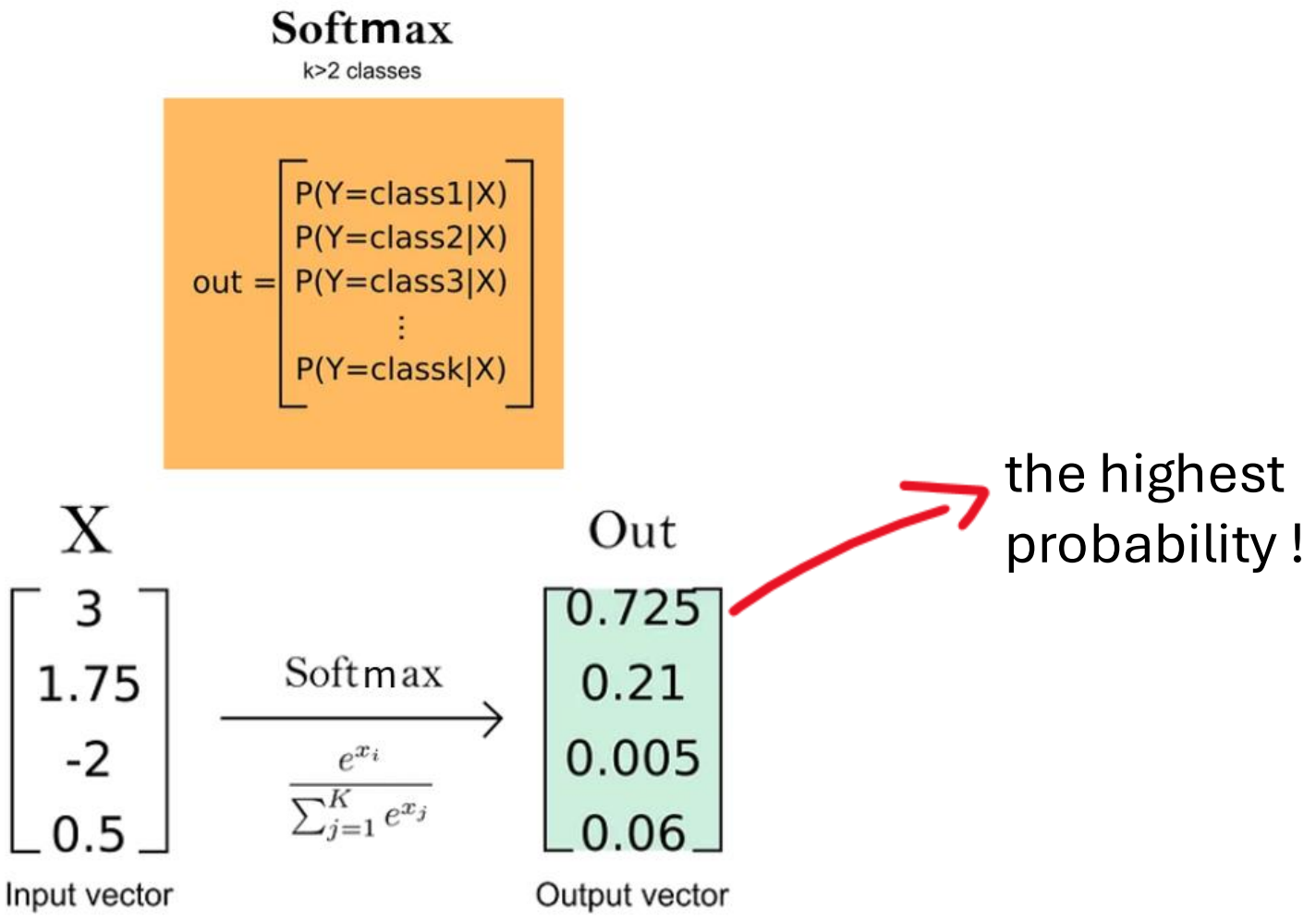
```
torch.Size([18, 1, 64, 64]) torch.float32
torch.Size([18]) torch.int64
```



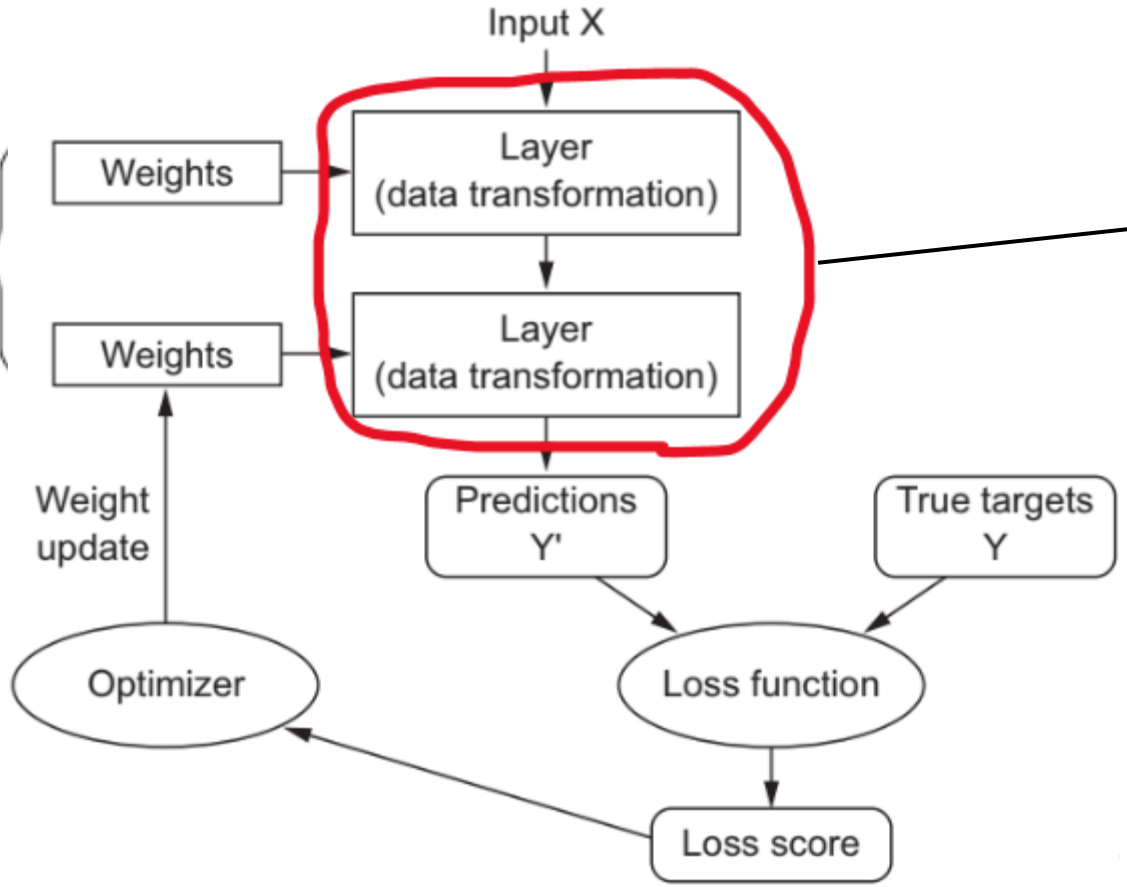
3) Softmax regression in PyTorch

- the goal of applying softmax regression to the FashionMNIST dataset is to **classify each image into one of 10 distinct clothing categories**

- It determines the probability of an image belonging to each class and assigns the label with the highest probability



Goal: finding the right values for these weights



DL model

Steps

- construct the Fashion-MNIST dataset and access it in mini-batches:

```python
def load_data_fashion_mnist(batch_size, resize=None):
    """Download the Fashion-MNIST dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(root="../data", train=False, transform=trans, download=True)
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [50000, 10000], generator=torch.Generator().manual_seed(42))

    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))

batch_size = 256
train_iter, val_iter, test_iter = load_data_fashion_mnist(batch_size)
```

- define the softmax regression model:

```python
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
```

we will discuss later about it

- 784 because we have 28×28 inputs (each image is flattened to a vector)
- 10 because we have 10 outputs (1 probability for each of the 10 classes)

- initialize the parameters:

```python
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights)
```

Steps

- specify the loss function to be used:

```
loss = nn.CrossEntropyLoss()
```

- specify the optimizer to be used:

```
lr = 0.1
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

- use the *evaluate_accuracy()* function which is already done in lab:

```
def evaluate_accuracy(net, data_iter):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Simple Example

Suppose a model classifies 100 images, and 90 of those classifications match the true labels. The accuracy would be:

$$\text{Accuracy} = \frac{90}{100} = 0.90 = 90\%$$

Steps

- use *train_epoch()* function which is already done in lab:

```python
def train_epoch(net, train_iter, loss, optimizer):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

→ **train the model for 1 epoch**

- use *train()* function which is already done in lab:

```python
def train(net, train_iter, val_iter, loss, num_epochs, optimizer):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')

    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

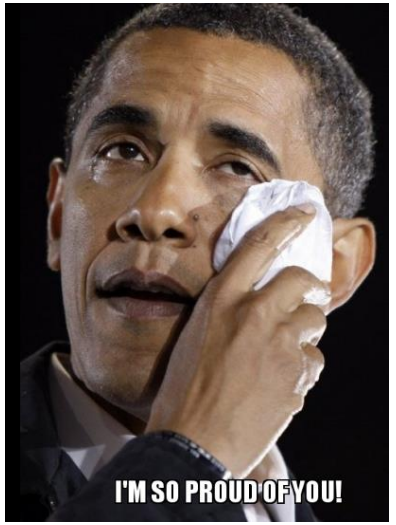
→ **train the model for multiple epochs, specified by *num_epochs***

Steps

- train the model for 10 epochs by running *train()* function:

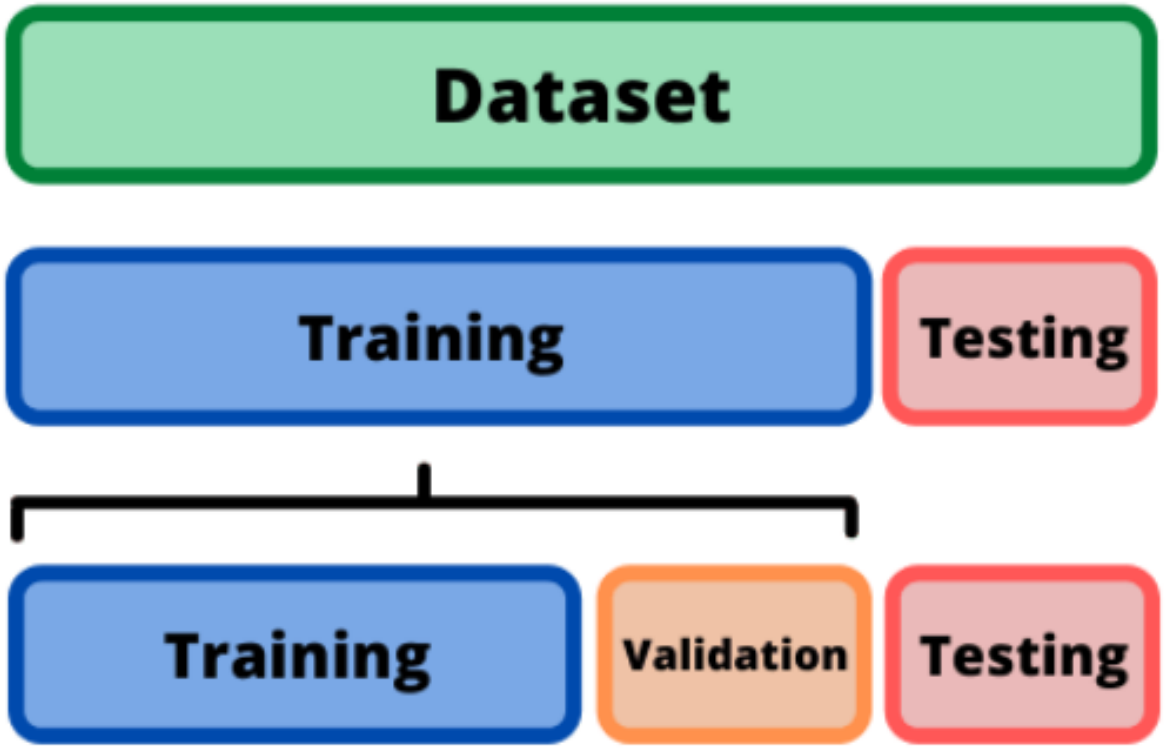
```
num_epochs = 10
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, loss, num_epochs, optimizer)
```

```
Epoch 1, Train loss 0.82, Train accuracy 73.86, Validation loss 0.63, Validation accuracy 78.73
Epoch 2, Train loss 0.58, Train accuracy 80.99, Validation loss 0.57, Validation accuracy 81.14
Epoch 3, Train loss 0.54, Train accuracy 82.23, Validation loss 0.53, Validation accuracy 82.20
Epoch 4, Train loss 0.51, Train accuracy 82.91, Validation loss 0.52, Validation accuracy 82.29
Epoch 5, Train loss 0.49, Train accuracy 83.39, Validation loss 0.51, Validation accuracy 82.79
Epoch 6, Train loss 0.48, Train accuracy 83.79, Validation loss 0.49, Validation accuracy 83.34
Epoch 7, Train loss 0.47, Train accuracy 84.01, Validation loss 0.51, Validation accuracy 82.04
Epoch 8, Train loss 0.46, Train accuracy 84.24, Validation loss 0.47, Validation accuracy 83.80
Epoch 9, Train loss 0.46, Train accuracy 84.63, Validation loss 0.49, Validation accuracy 83.52
Epoch 10, Train loss 0.45, Train accuracy 84.59, Validation loss 0.53, Validation accuracy 80.71
```



Training/validation/test split

- In ML/DL, datasets are typically divided into 3 parts: **training**, **validation**, and **test** sets (**in the first part we worked just with training and test !**)



Example Split

- **Training Set**: 70-80% of the total data
- **Validation Set**: 10-15% of the total data
- **Test Set**: 10-15% of the total data

Training/validation/test split

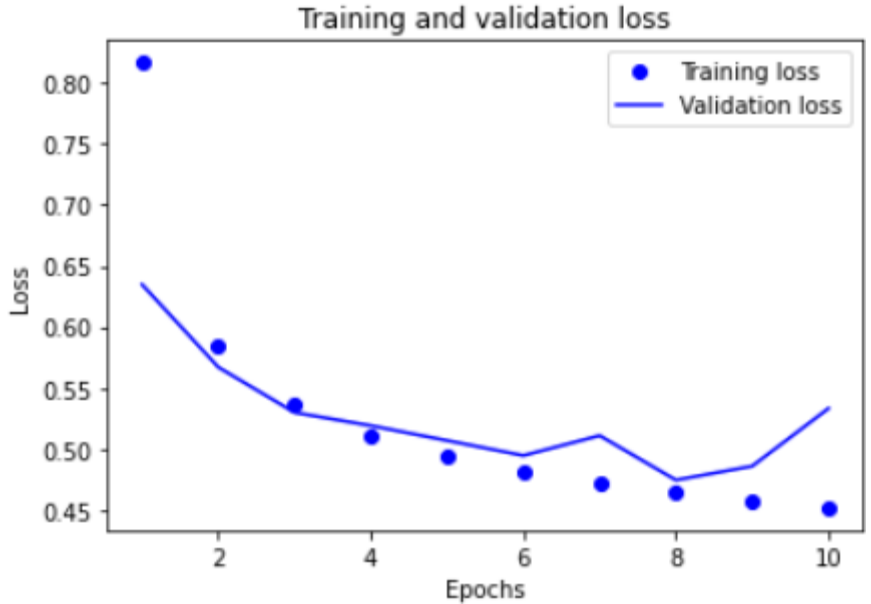
- **Training Set**
  - **Purpose**: is the subset of data used to train the ML/DL model.
  - **What Happens**: during training, the model adjusts its parameters (e.g., weights in neural networks) using algorithms like gradient descent to optimize a specific objective function, such as minimizing loss
  - **Size**: usually the largest portion of the dataset to ensure the model has enough data to learn from
- **Validation Set**
  - **Purpose**: is used to fine-tune the model and evaluate its performance during training. It acts as an independent dataset to monitor how well the model generalizes to unseen data.
  - **What Happens**:
    - it helps detect overfitting or underfitting
    - used to tune hyperparameters (e.g., learning rate, number of layers, dropout rate)
  - **Size**: typically a smaller portion of the dataset than the training set, often split from the original data
- **Test Set**
  - **Purpose**: is an entirely separate dataset used to assess the final performance of the trained model. It provides an unbiased evaluation of the model's ability to generalize to new data
  - **What Happens**:
    - it simulates how the model will perform in real-world scenarios
    - no training or fine-tuning should happen on the test set to ensure a fair evaluation
  - **Size**: similar in size to the validation set, but this depends on the size of the overall dataset

Inspecting the training

- plot the loss for the training and validation sets using the *plot_loss()* function:

```
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()

plot_loss(train_loss_all, val_loss_all)
```

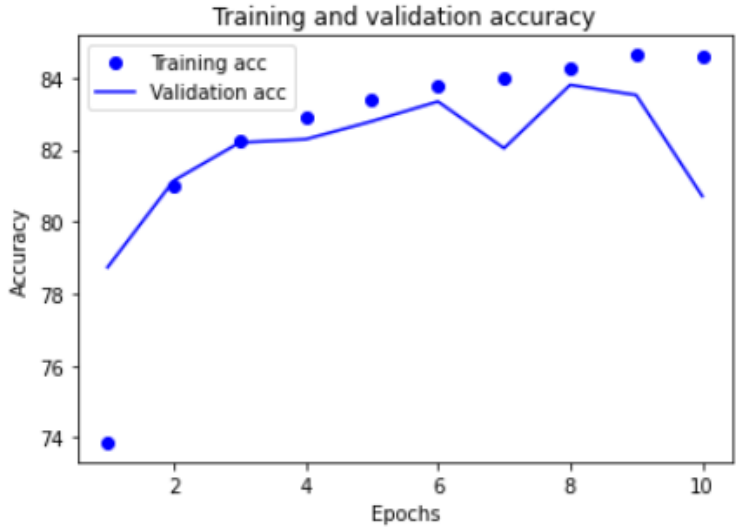


Inspecting the training

- plot the accuracy for the training and validation sets using the *plot_accuracy()* function:

```
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()

plot_accuracy(train_acc_all, val_acc_all)
```



Finally evaluate the model

- evaluate the trained model on the test set, which was not used at all during the training process:

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

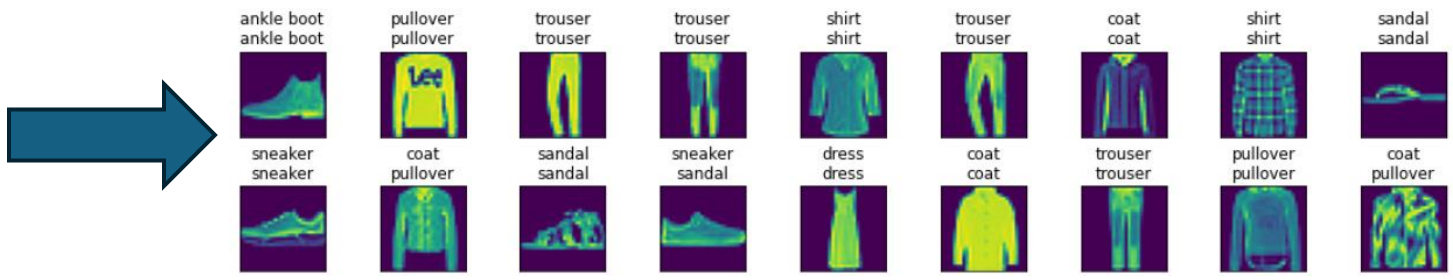
```
Test loss 0.55, Test accuracy 80.06
```

already done in lab

- we can also use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

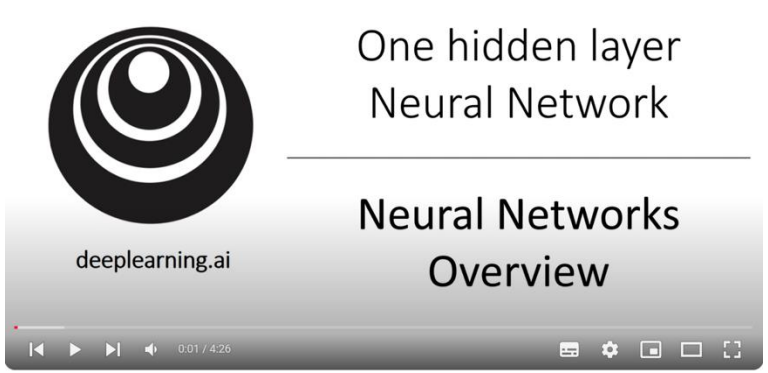
```
def predict(net, test_iter, n=18):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_fashion_mnist_labels(y)
    preds = get_fashion_mnist_labels(net(X).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].reshape((n, 28, 28)), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```

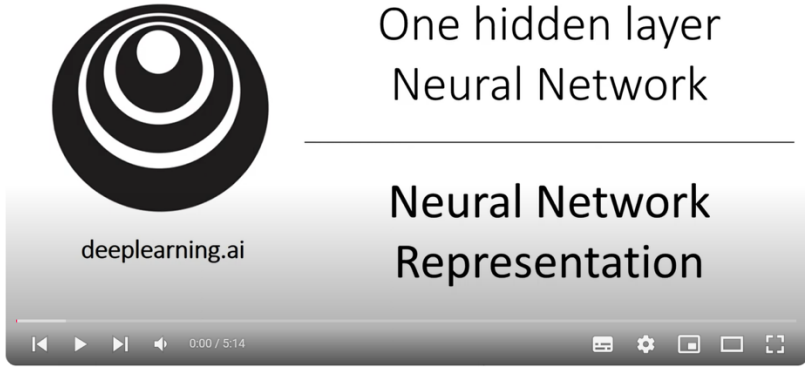


NOTE: *get_fashion_mnist_labels()* and *show_images()* functions are already done in FashionMNIST dataset section

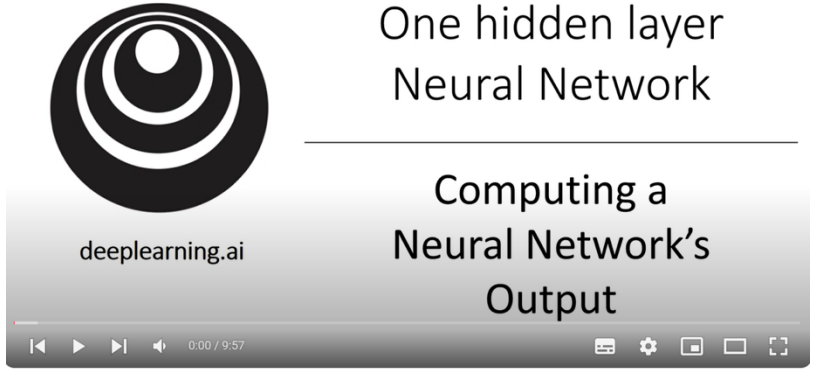
4) Multilayer perceptrons



Neural Network Overview (C1W3L01)

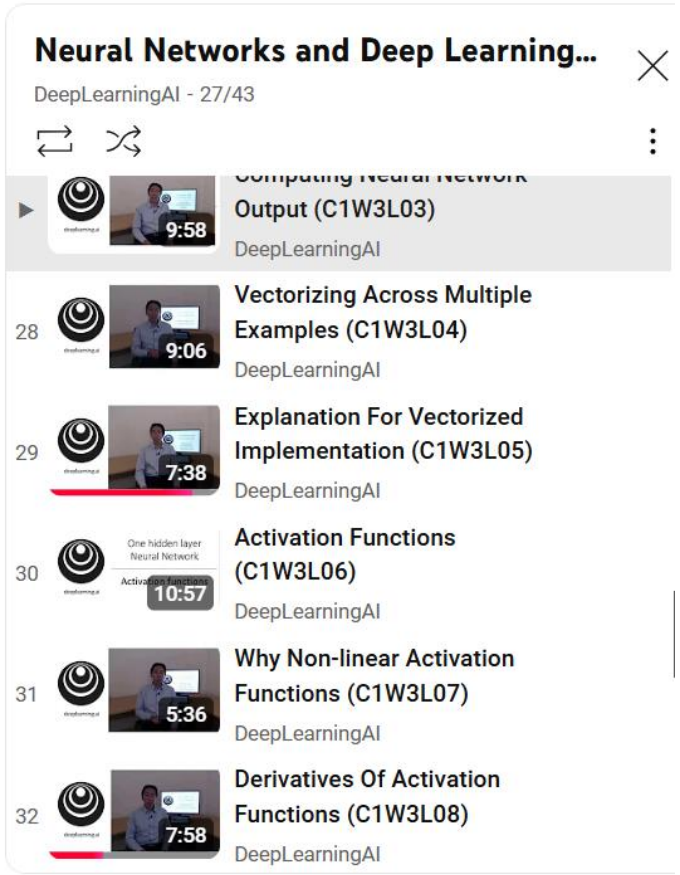


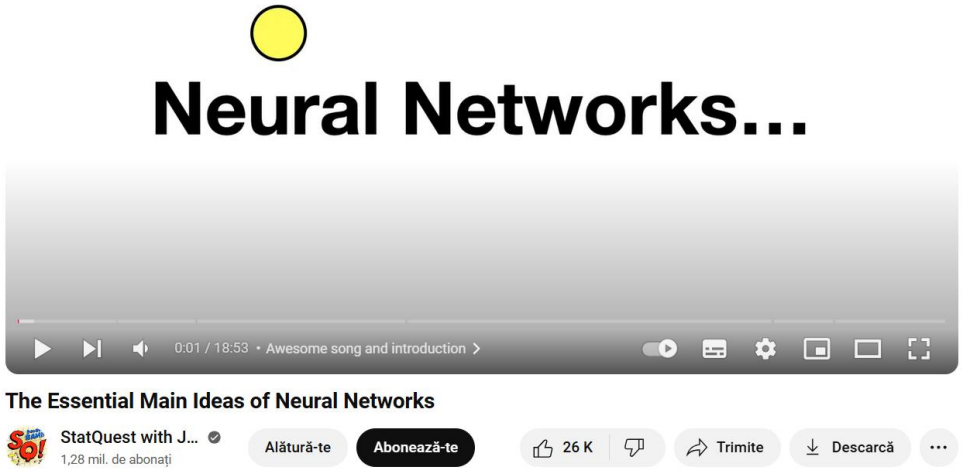
Neural Network Representations (C1W3L02)



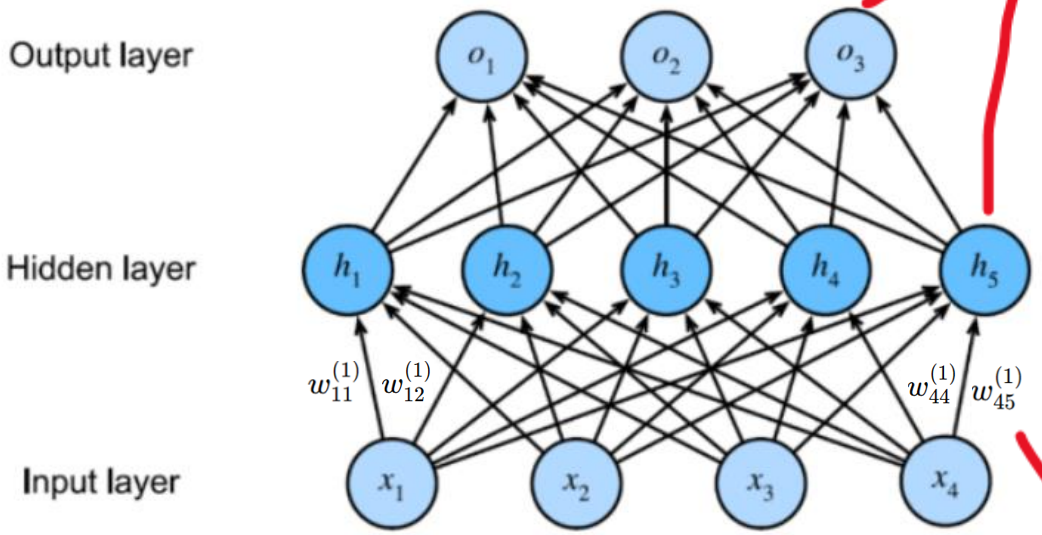
Computing Neural Network Output (C1W3L03)

...or easier:





A simple example



nodes/neurons

bias – usually we don't represent it in the diagrams

$b^{(1)}$

weights – the knowledge of the neural network

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(n)} & x_2^{(n)} & x_3^{(n)} & x_4^{(n)} \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} & w_{14}^{(1)} & w_{15}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} & w_{24}^{(1)} & w_{25}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} & w_{34}^{(1)} & w_{35}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} & w_{44}^{(1)} & w_{45}^{(1)} \end{bmatrix}$$

$$b^{(1)} = \begin{bmatrix} b_1^{(1)} & b_2^{(1)} & b_3^{(1)} & b_4^{(1)} & b_5^{(1)} \end{bmatrix}$$

$$h_1 = \sigma\left(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + w_{41}^{(1)}x_4 + b_1^{(1)}\right)$$

$$h_2 = \sigma\left(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{32}^{(1)}x_3 + w_{42}^{(1)}x_4 + b_2^{(1)}\right)$$

$$h_3 = \sigma\left(w_{13}^{(1)}x_1 + w_{23}^{(1)}x_2 + w_{33}^{(1)}x_3 + w_{43}^{(1)}x_4 + b_3^{(1)}\right)$$

$$h_4 = \sigma\left(w_{14}^{(1)}x_1 + w_{24}^{(1)}x_2 + w_{34}^{(1)}x_3 + w_{44}^{(1)}x_4 + b_4^{(1)}\right)$$

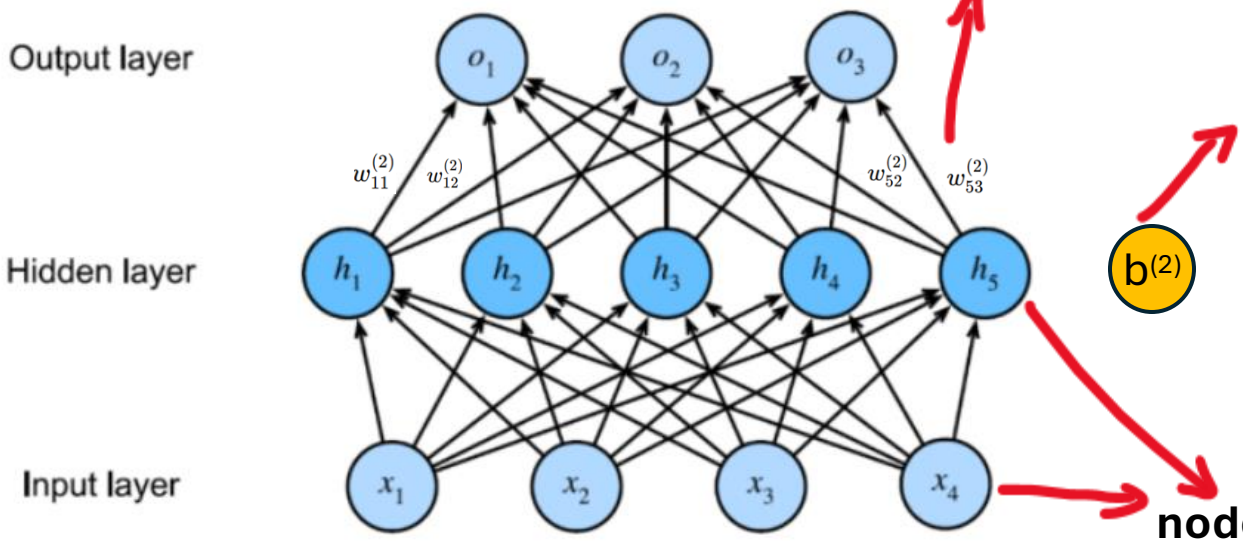
$$h_5 = \sigma\left(w_{15}^{(1)}x_1 + w_{25}^{(1)}x_2 + w_{35}^{(1)}x_3 + w_{45}^{(1)}x_4 + b_5^{(1)}\right)$$

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})$$

activation function (we will discuss about it in a moment)

A simple example

weights – the knowledge of the neural network



bias – usually we don't represent it in the diagrams

nodes/neurons

$$W^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} \\ w_{41}^{(2)} & w_{42}^{(2)} & w_{43}^{(2)} \\ w_{51}^{(2)} & w_{52}^{(2)} & w_{53}^{(2)} \end{bmatrix}$$

$$b^{(2)} = \begin{bmatrix} b_1^{(2)} & b_2^{(2)} & b_3^{(2)} \end{bmatrix}$$

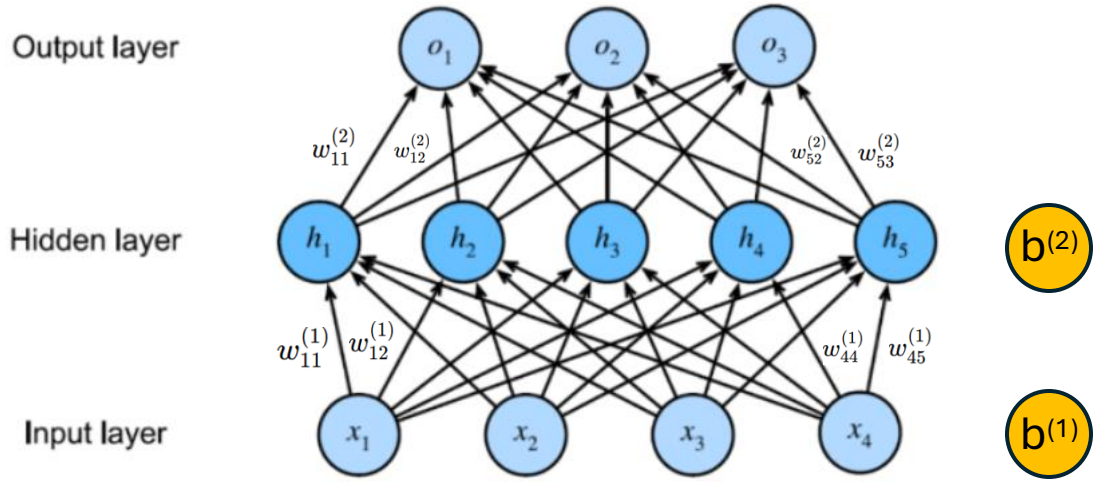
$$o_1 = w_{11}^{(2)} h_1 + w_{21}^{(2)} h_2 + w_{31}^{(2)} h_3 + w_{41}^{(2)} h_4 + w_{51}^{(2)} h_5 + b_1^{(2)}$$

$$o_2 = w_{12}^{(2)} h_1 + w_{22}^{(2)} h_2 + w_{32}^{(2)} h_3 + w_{42}^{(2)} h_4 + w_{52}^{(2)} h_5 + b_2^{(2)}$$

$$o_3 = w_{13}^{(2)} h_1 + w_{23}^{(2)} h_2 + w_{33}^{(2)} h_3 + w_{43}^{(2)} h_4 + w_{53}^{(2)} h_5 + b_3^{(2)}$$

$$\Leftrightarrow \quad \mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

A simple example



$$h_1 = \sigma\left(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + w_{41}^{(1)}x_4 + b_1^{(1)}\right)$$
$$h_2 = \sigma\left(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{32}^{(1)}x_3 + w_{42}^{(1)}x_4 + b_2^{(1)}\right)$$
$$h_3 = \sigma\left(w_{13}^{(1)}x_1 + w_{23}^{(1)}x_2 + w_{33}^{(1)}x_3 + w_{43}^{(1)}x_4 + b_3^{(1)}\right)$$
$$h_4 = \sigma\left(w_{14}^{(1)}x_1 + w_{24}^{(1)}x_2 + w_{34}^{(1)}x_3 + w_{44}^{(1)}x_4 + b_4^{(1)}\right)$$
$$h_5 = \sigma\left(w_{15}^{(1)}x_1 + w_{25}^{(1)}x_2 + w_{35}^{(1)}x_3 + w_{45}^{(1)}x_4 + b_5^{(1)}\right)$$

$$o_1 = w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + w_{31}^{(2)}h_3 + w_{41}^{(2)}h_4 + w_{51}^{(2)}h_5 + b_1^{(2)}$$
$$o_2 = w_{12}^{(2)}h_1 + w_{22}^{(2)}h_2 + w_{32}^{(2)}h_3 + w_{42}^{(2)}h_4 + w_{52}^{(2)}h_5 + b_2^{(2)}$$
$$o_3 = w_{13}^{(2)}h_1 + w_{23}^{(2)}h_2 + w_{33}^{(2)}h_3 + w_{43}^{(2)}h_4 + w_{53}^{(2)}h_5 + b_3^{(2)}$$

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

We want to find these !

Some notes

- the **number of neurons per hidden layer** and the **number of hidden layers**

⟹ ***hyperparameters*** that we need to decide about them

- some of the most used activation functions (σ):

$$ReLU(x) = max(x, 0)$$ *- rectified linear unit*

$$sigmoid(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

$$tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$ *- hyperbolic tangent*

Why should we use activation functions?

- Suppose we don't use the activation function σ:

$$\boldsymbol{H} = \sigma(\boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}$$



$$\boldsymbol{H} = \boldsymbol{X}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}$$
$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}$$



Substitute $H = XW^{(1)} + b^{(1)}$ into $O = HW^{(2)} + b^{(2)}$:

$$O = \left(XW^{(1)} + b^{(1)}\right) W^{(2)} + b^{(2)}$$



Why should we use activation functions?

$$O = \left(XW^{(1)}\right)W^{(2)} + \left(b^{(1)}\right)W^{(2)} + b^{(2)} = XW^{(1)}W^{(2)} + b^{(1)}W^{(2)} + b^{(2)} = XW + b$$

$W^{(1)}W^{(2)} = W$

$b^{(1)}W^{(2)} + b^{(2)} = b$

- **Conclusion**: this is **equivalent to a single-layer linear model**. As a result, no matter how many layers you stack, the model would be limited to learning **linear relationships** in the data

- activation functions introduce **non-linearity** into the model. By applying a non-linear activation function (like ReLU, sigmoid, or tanh) to *H*, the model can learn non-linear patterns

Example: the XOR problem



XOR

The XOR operation takes two binary inputs (x_1 and x_2) and produces an output (y) according to the following rule:

- $y = 1$ if $x_1 \neq x_2$ (inputs are different).
- $y = 0$ if $x_1 = x_2$ (inputs are the same).

XOR Truth Table:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Mathematically, a linear model computes:

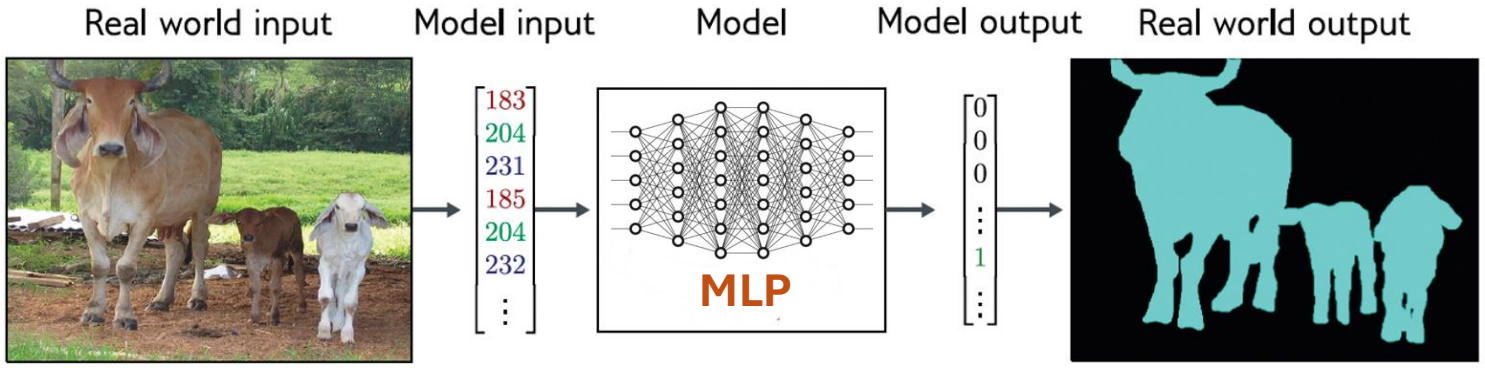
$$y = w_1 x_1 + w_2 x_2 + b$$

But no combination of w_1, w_2, b can satisfy the XOR truth table. Hence, XOR is **not linearly separable**.

A neural network with a hidden layer and **non-linear activation functions** can transform the input space to make XOR separable.

Example of problems

Image segmentation (semantic segmentation):



Depth estimation:

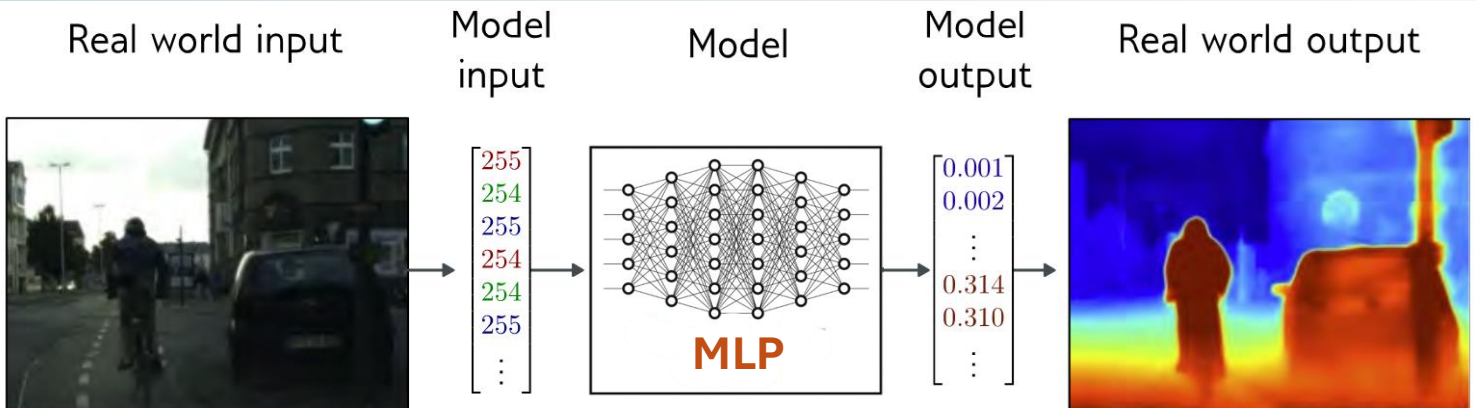
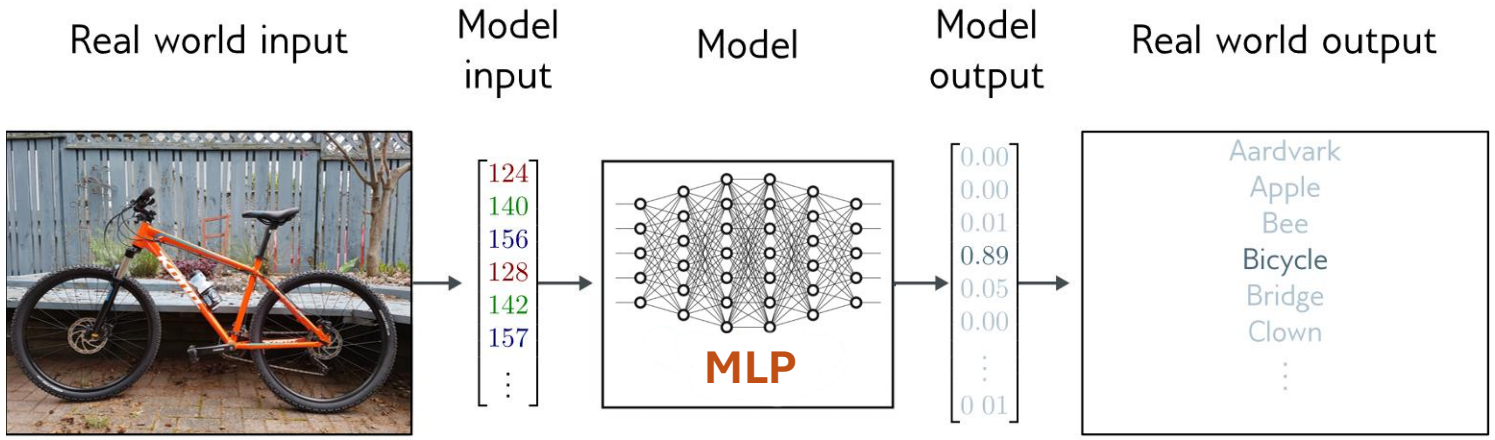
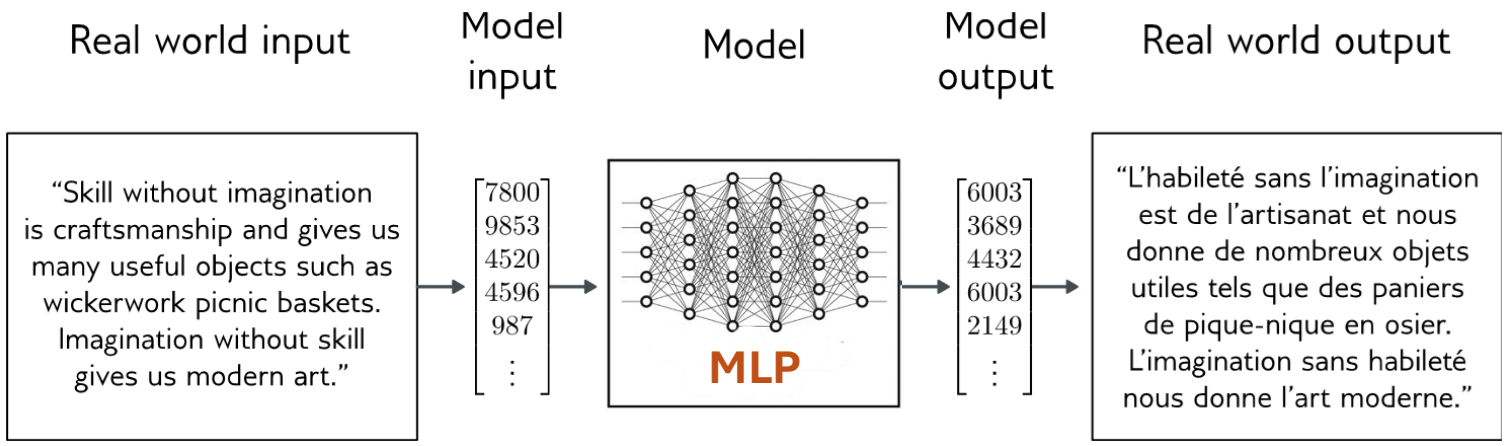


Image classification:

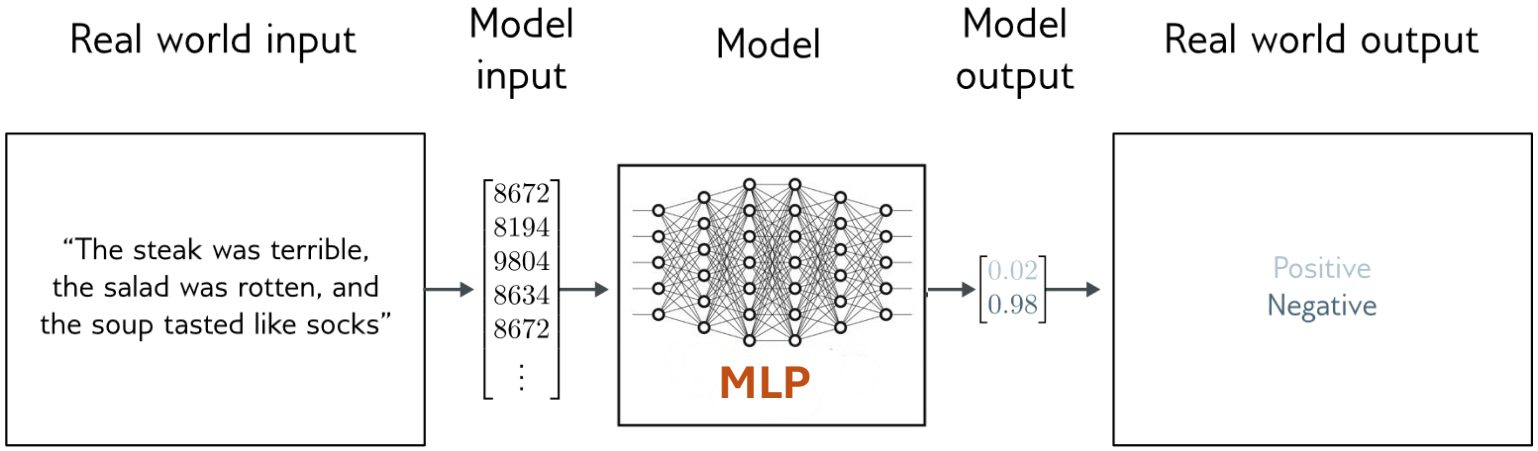


Example of problems

Translation:



Text classification:



NOTE

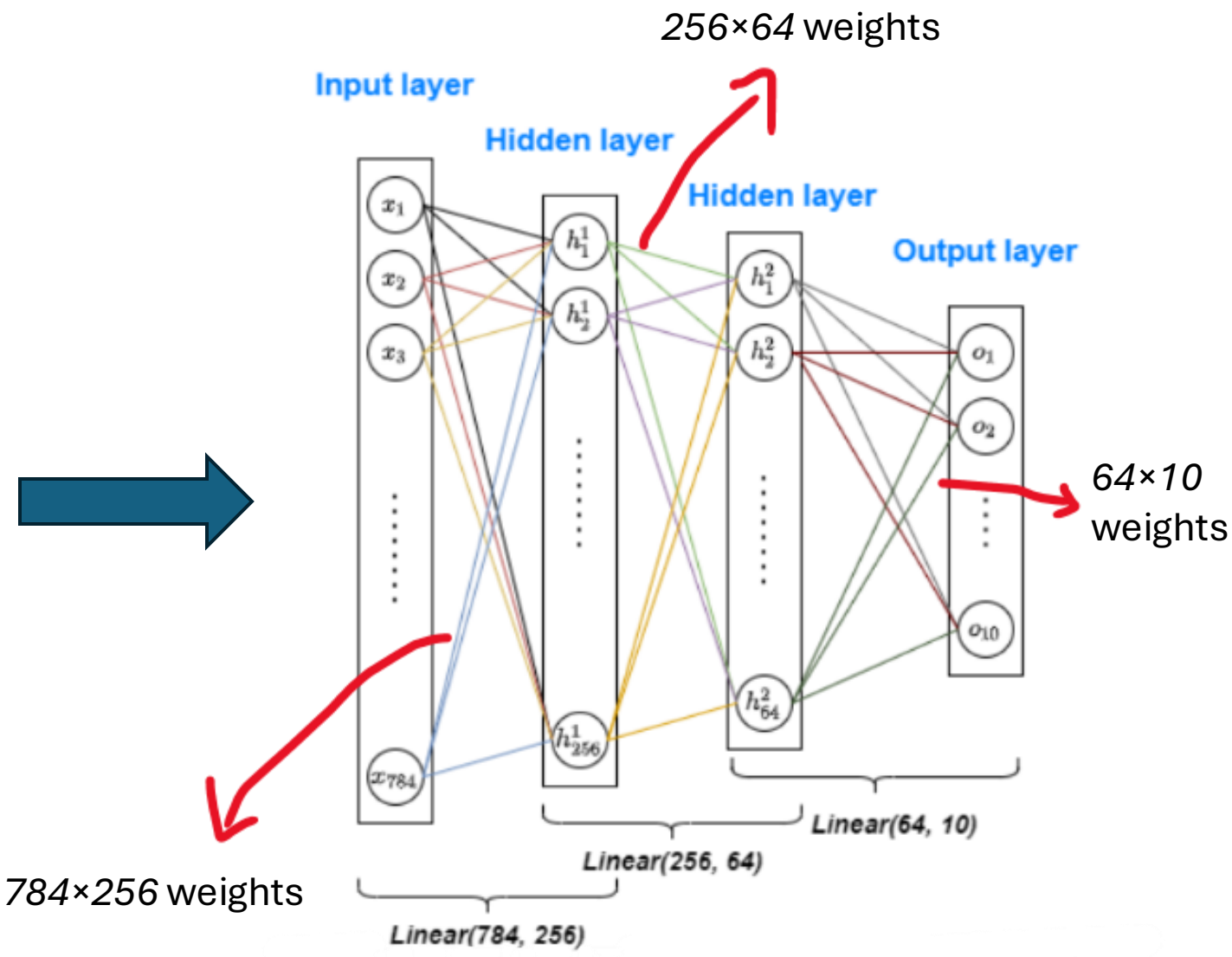
Even though the previously mentioned problems can be solved using MLPs, it is more appropriate to choose and use CNNs (for images) or RNNs (for text) to address them (as we will see in the next two labs) !

Code part

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 64),
                    nn.ReLU(),
                    nn.Linear(64, 10))

print(net)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): ReLU()
  (3): Linear(in_features=256, out_features=64, bias=True)
  (4): ReLU()
  (5): Linear(in_features=64, out_features=10, bias=True)
)
```



Softmax regression from the second section of the lab

```
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.55, Test accuracy 80.06
```

Softmax regression with an increased network

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 10))
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.49, Test accuracy 81.66
```

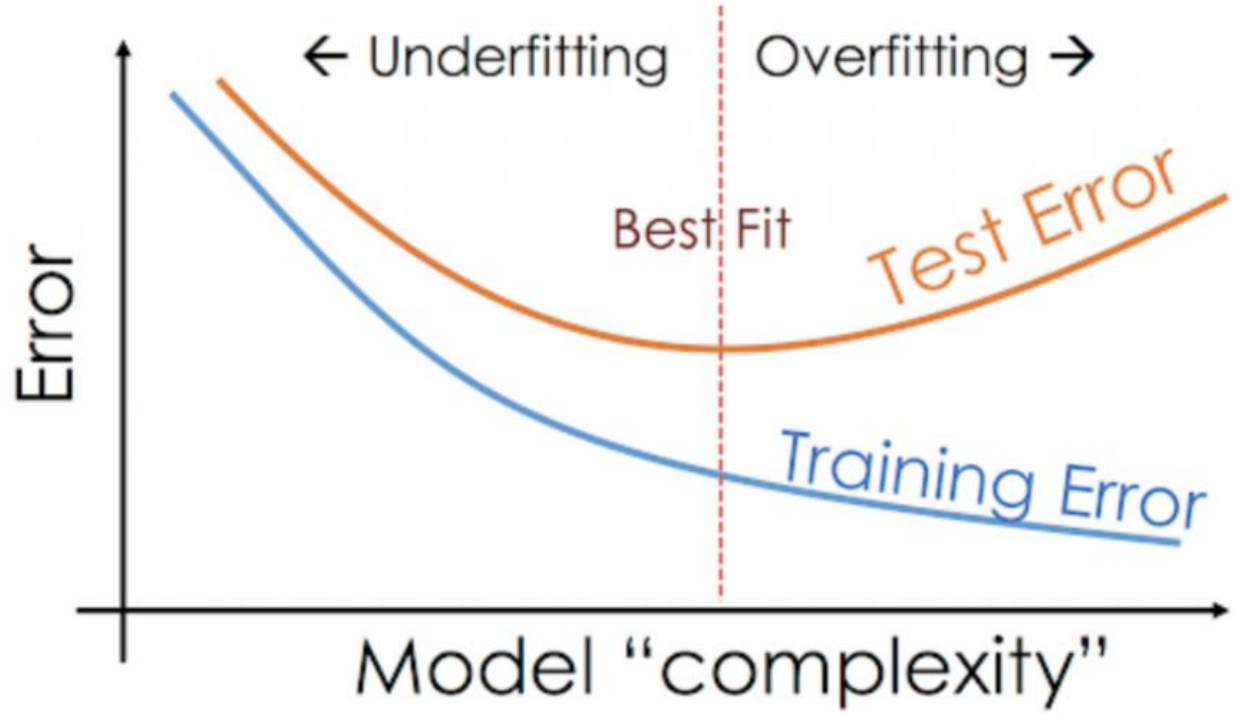


5) Weight decay

Kind reminder: overfitting vs underfitting

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It means the model does not fit the training data well and has high **bias** but low **variance**, resulting in poor performance on both training and test datasets.

Overfitting happens when a model is too complex and fits the training data "too well," even capturing noise and random fluctuations. As a result, it has low **bias** but high **variance**—it performs very well on the training data but poorly on unseen data because it lacks generalization

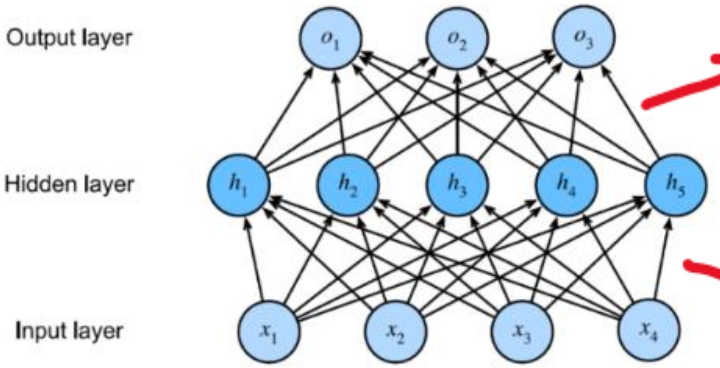


- it simply adds a penalty term to the loss function
- for example, consider the MSE loss given by:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

- for the model below, the "new" loss function will be:

penalty term



$\mathbf{W}^{(2)}$

$$\mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2),$$

where the Frobenius norm is defined as: $\|\mathbf{W}\|_F = \sqrt{Tr(\mathbf{W}^T\mathbf{W})}$

$\mathbf{W}^{(1)}$

Code part

- because **weight decay** is ubiquitous in neural network optimization, PyTorch makes it especially convenient, integrating weight decay into the optimization algorithm itself, for easy use in combination with any loss function:

```
wd = 0.001
optimizer = torch.optim.SGD(net.parameters(), weight_decay=wd, lr=lr)
```



that's all

Softmax regression from the second section of the lab

```
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
```

```
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.55, Test accuracy 80.06
```

Softmax regression with an increased network and **weight decay** applied

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 10))
```

```
wd = 0.001
optimizer = torch.optim.SGD(net.parameters(), weight_decay=wd, lr=lr)
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.45, Test accuracy 84.04
```



6) Dropout

Dropout steps

- **dropout** is a regularization technique commonly used in neural networks to prevent overfitting

- it works by randomly "dropping out" (setting to zero) a subset of neurons during training, effectively forcing the network to not overfit to specific neurons or weights

- **at each training iteration, for each hidden neuron, decide whether to "drop" it based on a predefined dropout rate, p** (a probability value between 0 and 1)

- for a dropout rate p, each neuron is retained with a probability $1 - p$. This decision is made independently for every neuron in the layer

- the neurons that are "dropped out" have their activations set to zero for that iteration.

Dropout steps

- example: if p =0.2, each neuron has an 80% chance of being retained and a 20% chance of being dropped

- to maintain the expected value of the layer's output unchanged, **the retained neurons' activations are scaled by 1/(1− p)**

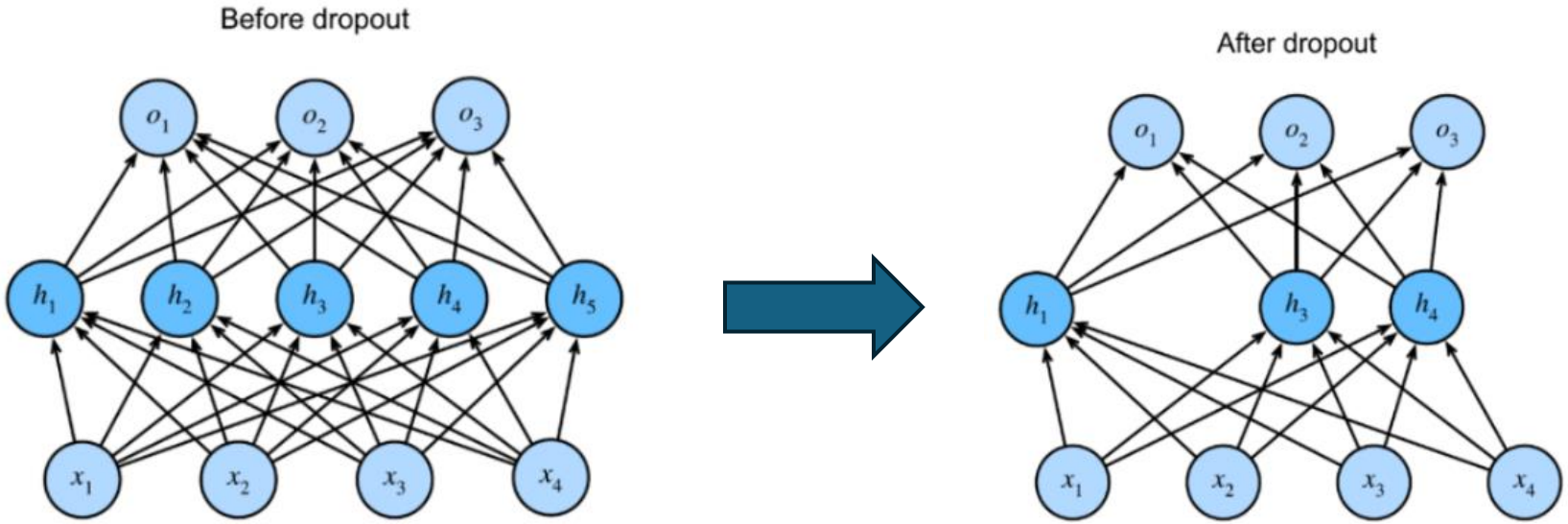
- **during inference, dropout is not applied**

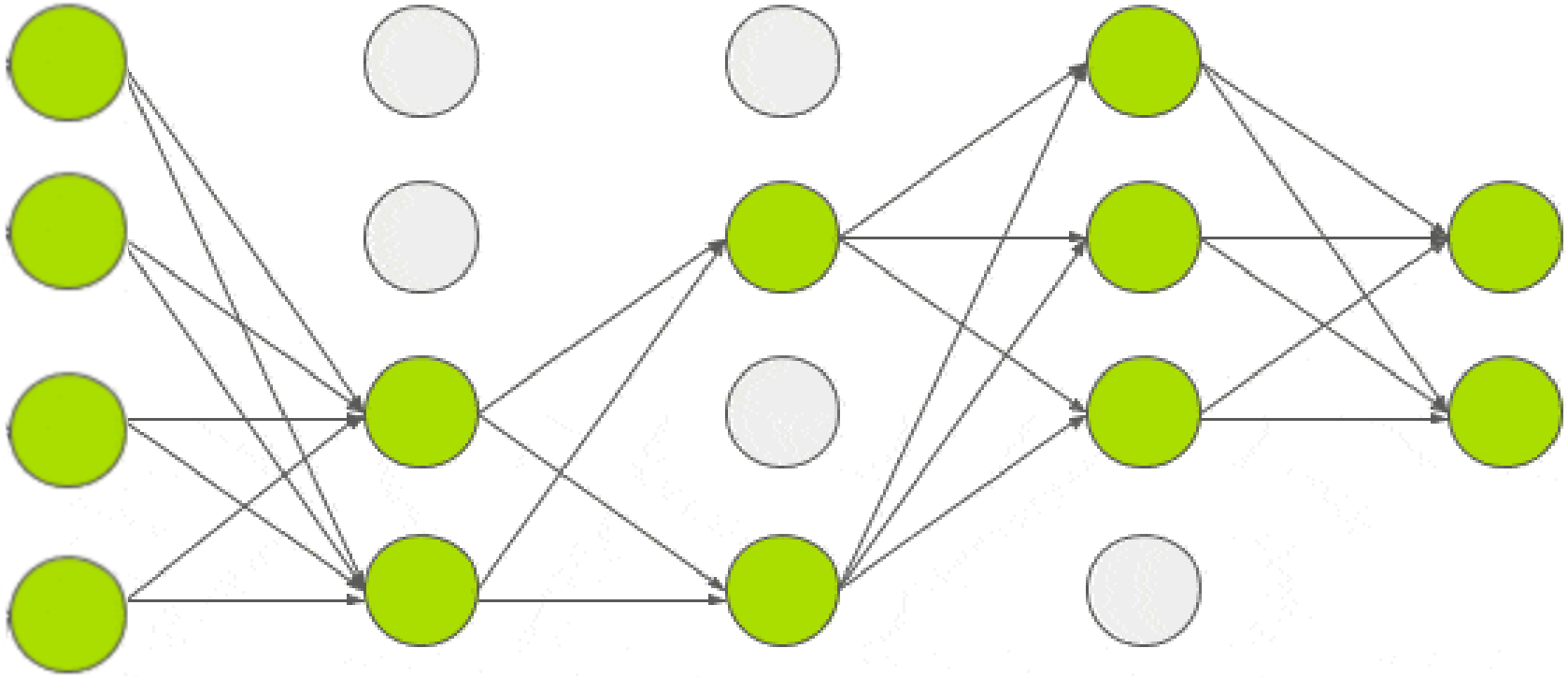
- **instead, the weights of all neurons are scaled down by (1− p).** This ensures that the output remains consistent with the effective behavior during training, where only a fraction of the neurons were active at a time

Example for Clarity:

1. **Training:**
   - A layer with 5 neurons has activations $[0.8, 0.5, 0.6, 0.4, 0.7]$.
   - Dropout rate $p = 0.4$ (60% retention).
   - After dropout, the retained neurons might be $[0.8, 0, 0.6, 0, 0.7]$ (randomly set to zero).
   - Remaining activations are scaled: $[0.8/(1-0.4), 0, 0.6/(1-0.4), 0, 0.7/(1-0.4)] = [1.33, 0, 1.0, 0, 1.17]$.

2. **Inference:**
   - Dropout is not applied.
   - Instead, weights are scaled down by $(1-p) = 0.6$, ensuring the expected value of outputs matches training.


Before dropout
After dropout
o_1
o_2
o_3
h_1
h_2
h_3
h_4
h_5
x_1
x_2
x_3
x_4



Code part

- all we need to do is add a ***nn.Dropout*** layer after each fully-connected layer, passing in the dropout probability as the only argument to its constructor

- during training, the ***nn.Dropout*** layer will randomly drop out outputs of the previous layer (or equivalently, the inputs to the subsequent layer) according to the specified dropout probability

- in the example below, we will drop 20% of the outputs of the first hidden layer, and 30% of the outputs of the second hidden layer

- when not in training mode, the ***nn.Dropout*** layer simply passes the data through, during testing

```
dropout1, dropout2 = 0.2, 0.3

net = nn.Sequential(nn.Flatten(),
        nn.Linear(784, 256),
        nn.ReLU(),
        # Add a dropout layer after the first fully connected layer
        nn.Dropout(dropout1),
        nn.Linear(256, 256),
        nn.ReLU(),
        # Add a dropout layer after the second fully connected layer
        nn.Dropout(dropout2),
        nn.Linear(256, 10))
```

Softmax regression from the second section of the lab

```
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.55, Test accuracy 80.06
```

Softmax regression with an increased network and **dropout** applied

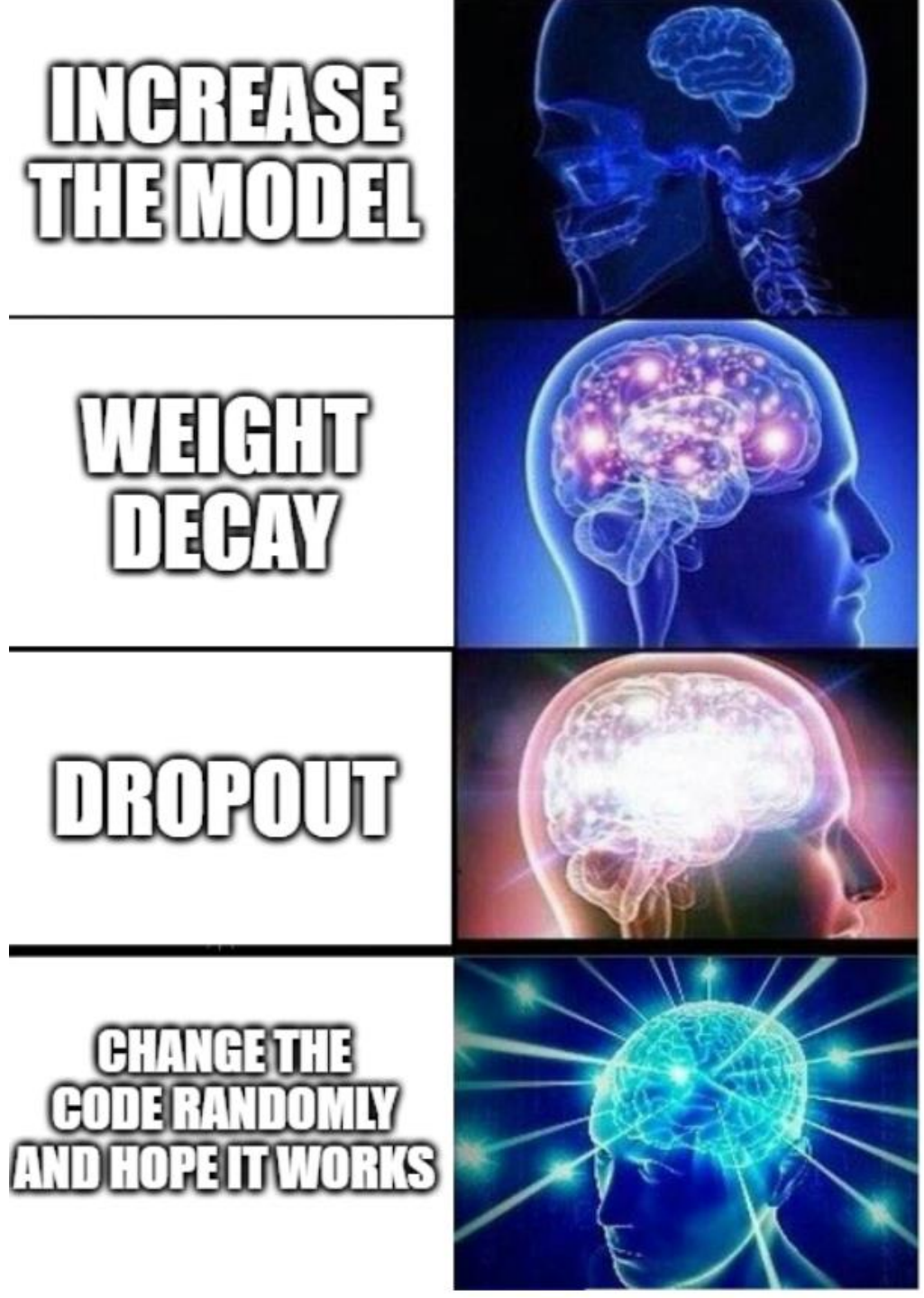
```
dropout1, dropout2 = 0.2, 0.3

net = nn.Sequential(nn.Flatten(),
        nn.Linear(784, 256),
        nn.ReLU(),
        # Add a dropout layer after the first fully connected layer
        nn.Dropout(dropout1),
        nn.Linear(256, 256),
        nn.ReLU(),
        # Add a dropout layer after the second fully connected layer
        nn.Dropout(dropout2),
        nn.Linear(256, 10))
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.43, Test accuracy 84.50
```




INCREASE THE MODEL
WEIGHT DECAY
DROPOUT
CHANGE THE CODE RANDOMLY AND HOPE IT WORKS

7) Deep learning computation

- if we deal with complex neural networks, we will need to work with **blocks (classes)**:

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)

    def forward(self, x):
        return self.output(nn.ReLU()(self.hidden(x)))

net = MLP()
X = torch.randn(size=(2, 20))
Y = net(X)
```

AICI E MIEZUL !

Here we implement the behaviour/logic of the network

- inspect the parameters (weights and biases – w and b) of the neural network:

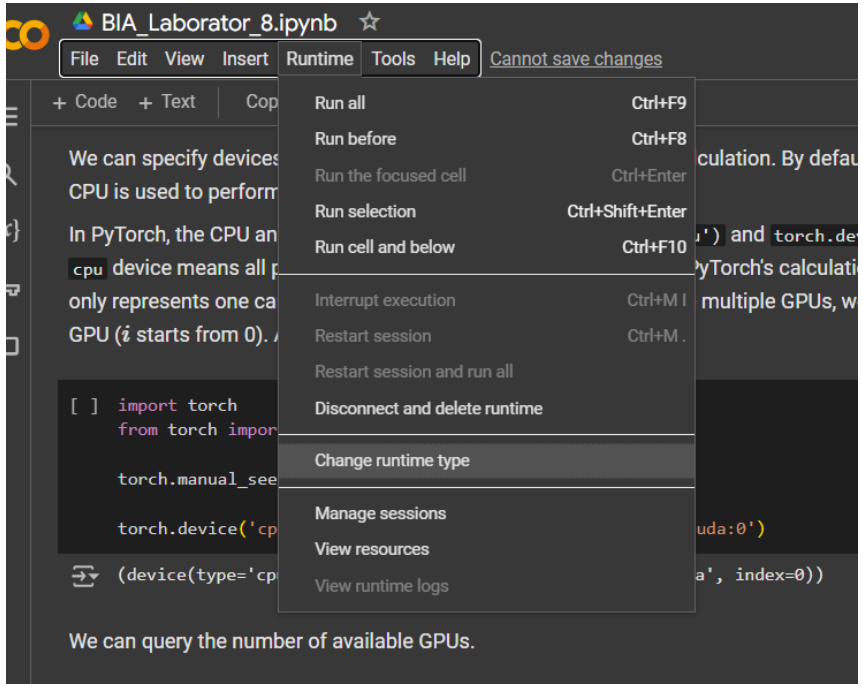
```
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 1))

print(net[0].state_dict()) # parameters of the first nn.Linear (w and b)
print(net[2].state_dict()) # parameters of the second nn.Linear (w and b)
```

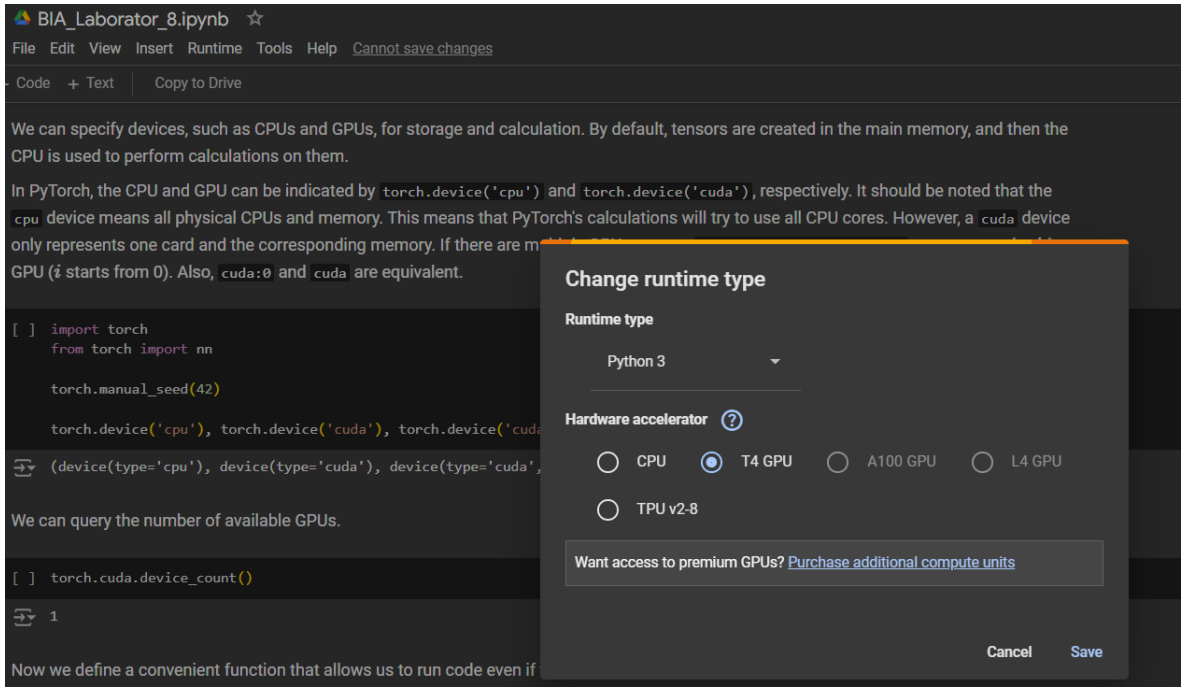
- put the model on the GPU if the GPU is available:

```python
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')

net = nn.Sequential(nn.Linear(3, 1))
net = net.to(device=try_gpu())
```







Old Exercises (same as for PI)

Laboratory 8 – Exercises

1. Estimate the following linear model $y = 1 \cdot X - 1 + \epsilon$, where $X \in \mathbb{R}^{60 \times 1}$ consists of generated values from -3 to 3 and ϵ obeys a normal distribution with mean 0 and standard deviation of 0.1. Train the linear regression model as follows:
   - ➢ set the weights and bias to 1;
   - ➢ set the batch size to 10;
   - ➢ **implement** the mean squared error loss;
   - ➢ **implement** the mini-batch gradient descent optimizer, $lr = 0.05$ (do not use the torch.optim package);
   - ➢ set the total number of iterations to 25;

   After the model is trained, compare the learned model parameters with the actual parameters.

2. Implement the softmax regression model in order to classify the images from the MNIST dataset (28×28 images of handwritten digits, 10 classes, labels are the digits from 0 to 9, 60000 training images and 10000 testing images) as follows:
   - ➢ set the batch size to 100;
   - ➢ the validation dataset consists of 30000 images and the rest represents the training dataset;
   - ➢ set the weights to 1;
   - ➢ use the cross-entropy loss function;
   - ➢ use mini-batch stochastic gradient descent with a learning rate of 0.15 as the optimization algorithm;
   - ➢ set the number of epochs to 15;
   - ➢ plot the accuracy for the training and validation sets;
   - ➢ use the trained model to classify 20 images.

   Evaluate the trained model on the test set.

3. Classify the MNIST dataset (28×28 images, 10 classes, 60000 training images and 10000 testing images) using an MLP as follows:
   - ➢ two hidden layers with 256 units;
   - ➢ add the hyperbolic tangent activation function and a dropout layer after each hidden layer (dropout probability of 40%);
   - ➢ use 45000 training images and 15000 validation images;
   - ➢ use the cross-entropy loss function;
   - ➢ apply Xavier initialization to the weights and set the biases to zero;
   - ➢ set the batch size to 512 and the learning rate to 0.15;
   - ➢ train the model for 10 epochs.

   Evaluate the trained model on the test set.

Solutions

1. Estimate the following linear model $y = 1 \cdot X - 1 + \epsilon$, where $X \in \mathbb{R}^{60\times 1}$ consists of generated values from -3 to 3 and ϵ obeys a normal distribution with mean 0 and standard deviation of 0.1. Train the linear regression model as follows:
   - ➢ set the weights and bias to 1;
   - ➢ set the batch size to 10;
   - ➢ **implement** the mean squared error loss;
   - ➢ **implement** the mini-batch gradient descent optimizer, $lr = 0.05$ (do not use the torch.optim package);
   - ➢ set the total number of iterations to 25;

   After the model is trained, compare the learned model parameters with the actual parameters.



go back to slide 11 to see the difference between epoch and iteration ☺

```python
import torch
import numpy as np

torch.manual_seed(42)
np.random.seed(42)

def synthetic_data(w, b, num_examples):
    """Generate y = Xw + b + noise."""
    X = torch.arange(-3, 3, 0.1).reshape(-1, 1) # Create X values from -3 to 3 with a step of 0.1
    y = X @ w + b # Compute the target values y = Xw + b
    y += torch.normal(0, 0.1, y.shape) # Add Gaussian noise with mean 0 and std dev 0.1
    return X, y.reshape(-1, 1)

true_w = torch.tensor([1.]) # True weight set to 1
true_b = -1 # True bias set to -1
features, labels = synthetic_data(true_w, true_b, 60) # Generate 60 data points
```

- the generated sequence will be: -3.0, -2.9, -2.8,...2.9
- it results in 60 values in total because the range spans from -3 to 2.9 with steps of 0.1

- `X = torch.linspace(-3, 3, steps=num_examples * len(w)).reshape(num_examples, len(w))` -> this is an alternative which creates a 1D tensor with steps evenly spaced between -3 and 3 (inclusive)

```python
from torch import nn   # `nn` is an abbreviation for neural networks

net = nn.Sequential(nn.Linear(1, 1)) # Single-layer linear model with 1 input and 1 output

net[0].weight.data.fill_(1)    # Set initial weight to 1
net[0].bias.data.fill_(1) # Set initial bias to 1

# Define the mean squared error loss function
def loss_fct(y_pred,y):
  return torch.mean((y_pred-y)**2) # Calculate average squared difference

# Parameters for mini-batch gradient descent
lr=0.05
num_iterations = 25  # Total number of parameter updates (iterations)

# Function to load data in mini-batches
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays) # Combine features and labels into a dataset
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train) # DataLoader for batching

# Create data iterator with batch size of 10
data_iter = load_array((features, labels), 10)
```

```python
# Training loop with exactly 25 iterations (parameter updates)
iteration = 0
while iteration < num_iterations:
    for X, y in data_iter:
        if iteration >= num_iterations:  # Stop after reaching the required number of iterations
            break
        l = loss_fct(net(X), y) # Compute the loss for the mini-batch
        l.backward() # Compute gradients of the loss with respect to model parameters

        # Manually update parameters without using an optimizer
        net[0].weight.data = net[0].weight.data - lr*net[0].weight.grad.data # Update weight
        net[0].bias.data = net[0].bias.data - lr*net[0].bias.grad.data # Update bias

        # Zero gradients after each batch to prevent accumulation
        net[0].weight.grad.data.zero_()
        net[0].bias.grad.data.zero_()

        iteration += 1  # Increment iteration count

    # Calculate and print the loss for the entire dataset periodically
    l = loss_fct(net(features), labels)
    print(f'Iteration {iteration}, Loss {l:f}')  # Display current iteration and loss
```

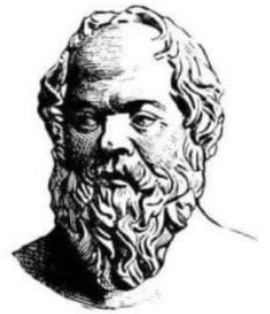
```
Iteration 6, Loss 1.136505
Iteration 12, Loss 0.320594
Iteration 18, Loss 0.097474
Iteration 24, Loss 0.034829
Iteration 25, Loss 0.029286
```

```python
print(net[0].weight.data)
print(net[0].bias.data)
```

```
tensor([[0.9943]])
tensor([-0.8434])
```

$$y = 1 \cdot X - 1 + \epsilon$$

"ehh, good enough"



- Mediocrates

NOTE: if we had specified **epochs** instead of **iterations** in the statement we would have had the following solution:

```
from torch import nn   # `nn` is an abbreviation for neural networks

net = nn.Sequential(nn.Linear(1, 1)) # Single-layer linear model with 1 input and 1 output

net[0].weight.data.fill_(1) # Set initial weight to 1
net[0].bias.data.fill_(1) # Set initial bias to 1

# Define the mean squared error loss function
def loss_fct(y_pred,y):
    return torch.mean((y_pred-y)**2) # Calculate average squared difference

# Parameters for mini-batch gradient descent
lr=0.05
num_epochs = 25

# Function to load data in mini-batches
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays) # Combine features and labels into a dataset
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train) # DataLoader for batching

# Create data iterator with batch size of 10
data_iter = load_array((features, labels), 10)
```

```
# Training loop with 25 epochs
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss_fct(net(X), y) # Compute the loss for the mini-batch
        l.backward() # Compute gradients of the loss with respect to model parameters

        # Manually update parameters without using an optimizer
        net[0].weight.data = net[0].weight.data - lr*net[0].weight.grad.data # Update weight
        net[0].bias.data = net[0].bias.data - lr*net[0].bias.grad.data # Update bias

        # Zero gradients after each batch to prevent accumulation
        net[0].weight.grad.data.zero_()
        net[0].bias.grad.data.zero_()

    # Calculate and print the loss for the entire dataset periodically
    l = loss_fct(net(features), labels)
    print(f'Epoch {epoch + 1}, Loss {l:f}')
```



```
# Print the final values of the weight and bias
print(net[0].weight.data) # Print learned weight
print(net[0].bias.data) # Print learned bias
```

```
tensor([[0.9999]])
tensor([-0.9786])
```

$$y = 1 \cdot X - 1 + \epsilon$$

```
Epoch 1, Loss 1.115737
Epoch 2, Loss 0.320603
Epoch 3, Loss 0.097224
Epoch 4, Loss 0.033696
Epoch 5, Loss 0.017093
Epoch 6, Loss 0.012058
Epoch 7, Loss 0.010675
Epoch 8, Loss 0.010128
Epoch 9, Loss 0.010006
Epoch 10, Loss 0.010035
Epoch 11, Loss 0.009948
Epoch 12, Loss 0.009964
Epoch 13, Loss 0.010335
Epoch 14, Loss 0.010146
Epoch 15, Loss 0.009956
Epoch 16, Loss 0.009959
Epoch 17, Loss 0.010083
Epoch 18, Loss 0.009949
Epoch 19, Loss 0.010089
Epoch 20, Loss 0.009955
Epoch 21, Loss 0.010074
Epoch 22, Loss 0.010018
Epoch 23, Loss 0.009983
Epoch 24, Loss 0.009943
Epoch 25, Loss 0.010032
```

2. Implement the softmax regression model in order to classify the images from the MNIST dataset (28×28 images of handwritten digits, 10 classes, labels are the digits from 0 to 9, 60000 training images and 10000 testing images) as follows:

- ➢ set the batch size to 100;
- ➢ the validation dataset consists of 30000 images and the rest represents the training dataset;
- ➢ set the weights to 1;
- ➢ use the cross-entropy loss function;
- ➢ use mini-batch stochastic gradient descent with a learning rate of 0.15 as the optimization algorithm;
- ➢ set the number of epochs to 15;
- ➢ plot the accuracy for the training and validation sets;
- ➢ use the trained model to classify 20 images.

Evaluate the trained model on the test set.

```python
import torchvision
import torch
from torch import nn
from torchvision import transforms
```

- construct the MNIST dataset and access it in mini-batches:

```python
def load_data_mnist(batch_size, resize=None):
    """Download the MNIST dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.MNIST(root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.MNIST(root="../data", train=False, transform=trans, download=True)
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [30000, 30000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))

# Set the batch size to 100 as specified
batch_size = 100
train_iter, val_iter, test_iter = load_data_mnist(batch_size, resize=None)
```

- define the softmax regression model:

```
# Define a simple neural network model with a flatten layer and a linear (fully connected) layer
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10)) # 784 inputs (28x28), 10 outputs (classes)
```

- initialize the parameters:

```
# Initialize the weights of the linear layer to 1 as per problem requirements
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight,1) # Set all weights to 1

net.apply(init_weights) # Apply the initialization to the model
```

- specify the loss function to be used:

```
# Use cross-entropy loss for multi-class classification
loss = nn.CrossEntropyLoss()
```

- specify the optimizer to be used:

```
# Define a stochastic gradient descent optimizer with learning rate 0.15
lr = 0.15
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

- use the *evaluate_accuracy()* function which is already done in lab:

```python
def evaluate_accuracy(net, data_iter):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Simple Example

Suppose a model classifies 100 images, and 90 of those classifications match the true labels. The accuracy would be:

$$\text{Accuracy} = \frac{90}{100} = 0.90 = 90\%$$

- use *train_epoch()* function which is already done in lab:

```python
def train_epoch(net, train_iter, loss, optimizer):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```



train the model for 1 epoch

- use *train()* function which is already done in lab:

```python
def train(net, train_iter, val_iter, loss, num_epochs, optimizer):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')

    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```



train the model for multiple epochs, specified by *num_epochs*

- train the model for 15 epochs by running *train()* function:

```
num_epochs = 15
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, loss, num_epochs, optimizer)
```

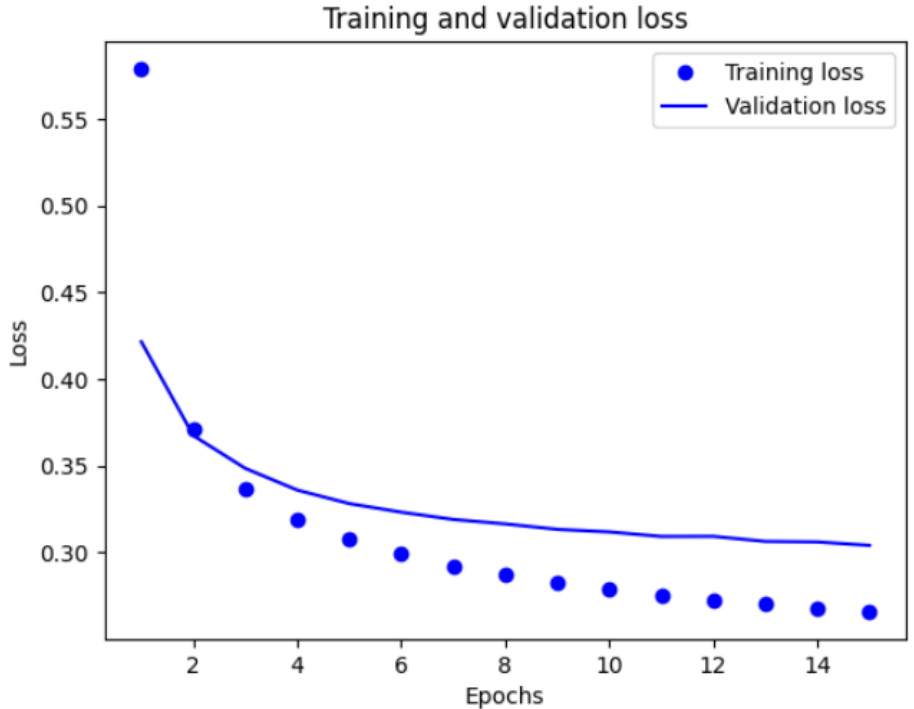
```
Epoch 1, Train loss 0.58, Train accuracy 85.79, Validation loss 0.42, Validation accuracy 88.30
Epoch 2, Train loss 0.37, Train accuracy 89.77, Validation loss 0.37, Validation accuracy 89.73
Epoch 3, Train loss 0.34, Train accuracy 90.59, Validation loss 0.35, Validation accuracy 90.26
Epoch 4, Train loss 0.32, Train accuracy 91.03, Validation loss 0.34, Validation accuracy 90.55
Epoch 5, Train loss 0.31, Train accuracy 91.36, Validation loss 0.33, Validation accuracy 90.82
Epoch 6, Train loss 0.30, Train accuracy 91.65, Validation loss 0.32, Validation accuracy 90.92
Epoch 7, Train loss 0.29, Train accuracy 91.87, Validation loss 0.32, Validation accuracy 90.97
Epoch 8, Train loss 0.29, Train accuracy 91.99, Validation loss 0.32, Validation accuracy 91.03
Epoch 9, Train loss 0.28, Train accuracy 92.17, Validation loss 0.31, Validation accuracy 91.05
Epoch 10, Train loss 0.28, Train accuracy 92.18, Validation loss 0.31, Validation accuracy 91.12
Epoch 11, Train loss 0.28, Train accuracy 92.40, Validation loss 0.31, Validation accuracy 91.14
Epoch 12, Train loss 0.27, Train accuracy 92.46, Validation loss 0.31, Validation accuracy 91.27
Epoch 13, Train loss 0.27, Train accuracy 92.51, Validation loss 0.31, Validation accuracy 91.33
Epoch 14, Train loss 0.27, Train accuracy 92.50, Validation loss 0.31, Validation accuracy 91.33
Epoch 15, Train loss 0.27, Train accuracy 92.65, Validation loss 0.30, Validation accuracy 91.40
```



- plot the loss for the training and validation sets using the *plot_loss()* function:

```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()

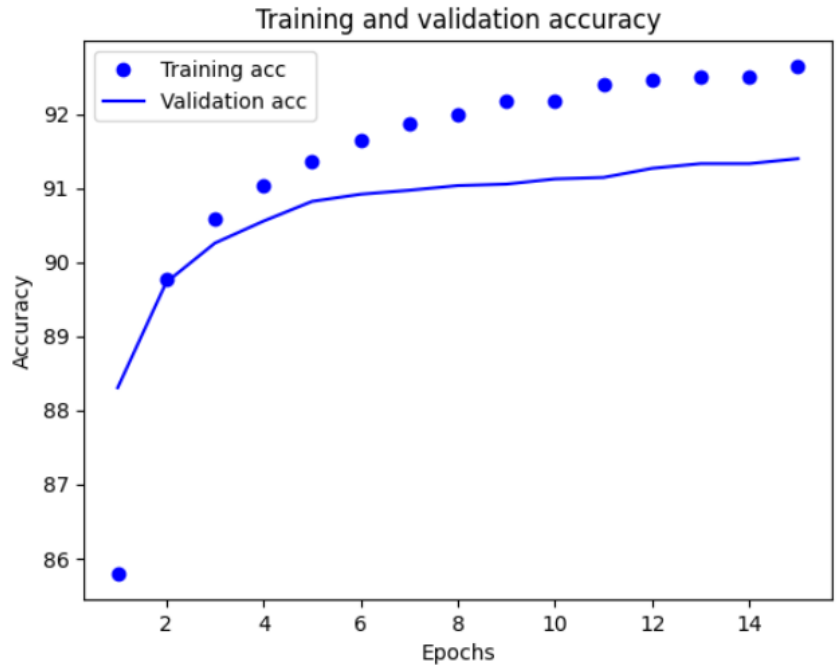
plot_loss(train_loss_all, val_loss_all)
```



- plot the accuracy for the training and validation sets using the *plot_accuracy()* function:

```
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()

plot_accuracy(train_acc_all, val_acc_all)
```



- evaluate the trained model on the test set, which was not used at all during the training process:

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.28, Test accuracy 92.15
```

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

```
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```



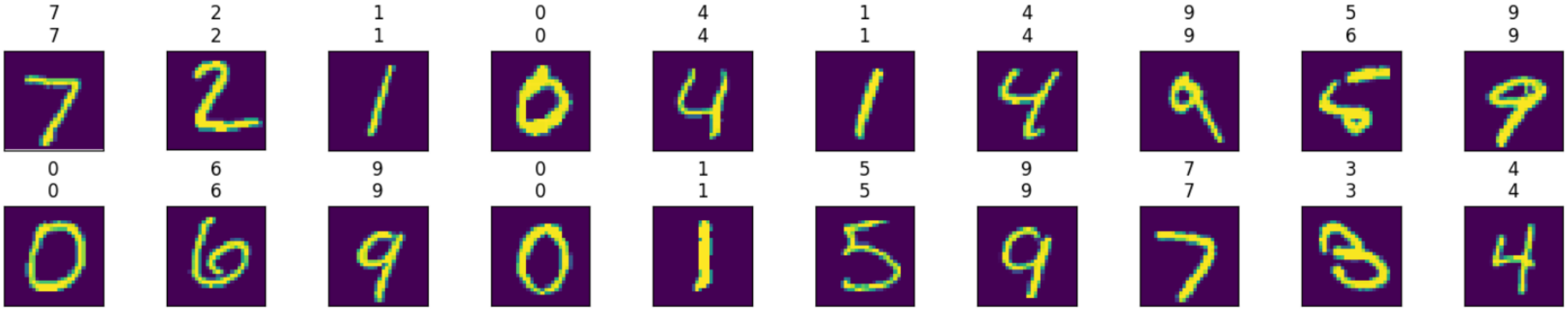
function already done in lab at FashionMNIST dataset section (slide 14)

```python
def get_mnist_labels(labels):
    """Return text labels for the MNIST dataset."""
    text_labels = ['0', '1', '2', '3', '4','5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

adaptation of *get_fashion_mnist_labels()* function from FashionMNIST dataset section (slide 14)

```python
def predict(net, test_iter, n=20):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_mnist_labels(y)
    preds = get_mnist_labels(net(X).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].reshape((n, 28, 28)), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```



3. Classify the MNIST dataset (28×28 images, 10 classes, 60000 training images and 10000 testing images) using an MLP as follows:
   - ➢ two hidden layers with 256 units;
   - ➢ add the hyperbolic tangent activation function and a dropout layer after each hidden layer (dropout probability of 40%);
   - ➢ use 45000 training images and 15000 validation images;
   - ➢ use the cross-entropy loss function;
   - ➢ apply Xavier initialization to the weights and set the biases to zero;
   - ➢ set the batch size to 512 and the learning rate to 0.15;
   - ➢ train the model for 10 epochs.

   Evaluate the trained model on the test set.

```python
import torchvision
import torch
from torch import nn
from torchvision import transforms
```

- construct the MNIST dataset and access it in mini-batches:

```python
def load_data_mnist(batch_size, resize=None):
    """Download the MNIST dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.MNIST(root="../data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.MNIST(root="../data", train=False, transform=trans, download=True)
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [45000, 15000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))

# Set the batch size to 100 as specified
batch_size = 100
train_iter, val_iter, test_iter = load_data_mnist(batch_size, resize=None)
```

- define the softmax regression model:

```
# Define the neural network with 2 hidden layers, tanh activation, and dropout layers
net = nn.Sequential(
    nn.Flatten(),                # Flatten the input tensor from (batch_size, 28, 28) to (batch_size, 784)
    nn.Linear(784, 256),         # First hidden layer with 256 units
    nn.Tanh(),                   # Apply hyperbolic tangent activation function
    nn.Dropout(0.4),             # Apply dropout with a probability of 40%
    nn.Linear(256, 256),         # Second hidden layer with 256 units
    nn.Tanh(),                   # Apply hyperbolic tangent activation function
    nn.Dropout(0.4),             # Apply dropout with a probability of 40%
    nn.Linear(256, 10)           # Output layer with 10 units (10 classes)
)
```

- initialize the parameters:

```
# Function to initialize the weights with Xavier initialization and set biases to zero
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)  # Xavier initialization for weights
        nn.init.constant_(m.bias, 0)      # Set biases to zero
```

- specify the loss function to be used:

```
# Use cross-entropy loss for multi-class classification
loss = nn.CrossEntropyLoss()
```

- specify the optimizer to be used:

```
# Define a stochastic gradient descent optimizer with learning rate 0.15
lr = 0.15
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

- use the *evaluate_accuracy()* function which is already done in lab:

```python
def evaluate_accuracy(net, data_iter):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Simple Example

Suppose a model classifies 100 images, and 90 of those classifications match the true labels. The accuracy would be:

$$\text{Accuracy} = \frac{90}{100} = 0.90 = 90\%$$

- use *train_epoch()* function which is already done in lab:

```python
def train_epoch(net, train_iter, loss, optimizer):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```



train the model for 1 epoch

- use *train()* function which is already done in lab:

```python
def train(net, train_iter, val_iter, loss, num_epochs, optimizer):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')

    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```



train the model for multiple epochs, specified by *num_epochs*

- train the model for 15 epochs by running *train()* function:

```
num_epochs = 15
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, loss, num_epochs, optimizer)
```

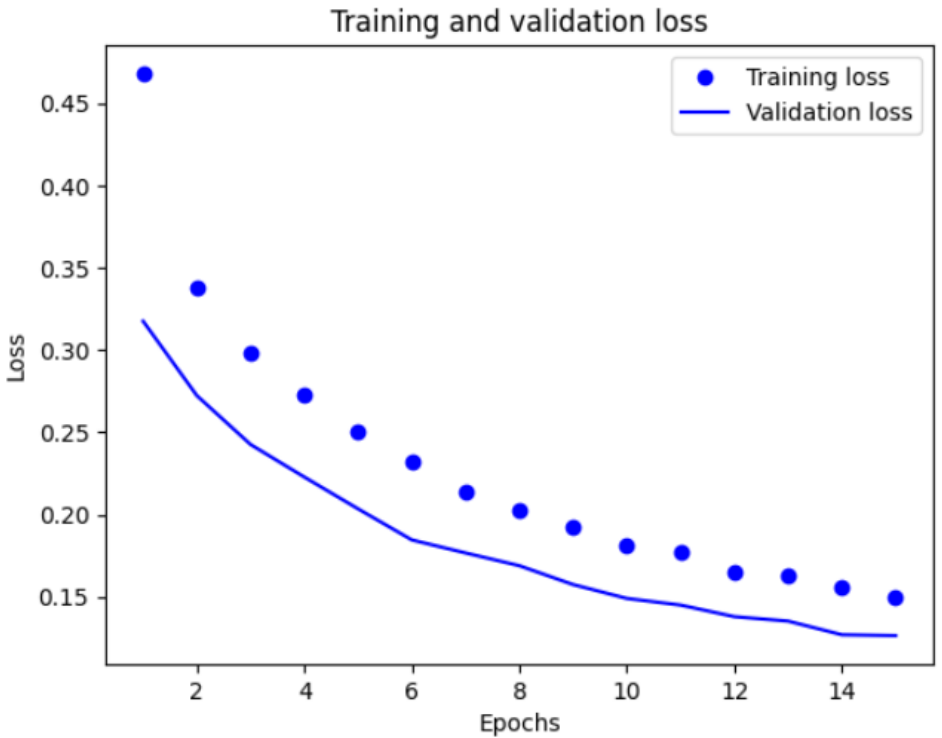
```
Epoch 1, Train loss 0.47, Train accuracy 85.88, Validation loss 0.32, Validation accuracy 90.82
Epoch 2, Train loss 0.34, Train accuracy 90.00, Validation loss 0.27, Validation accuracy 92.10
Epoch 3, Train loss 0.30, Train accuracy 91.19, Validation loss 0.24, Validation accuracy 92.84
Epoch 4, Train loss 0.27, Train accuracy 91.95, Validation loss 0.22, Validation accuracy 93.57
Epoch 5, Train loss 0.25, Train accuracy 92.47, Validation loss 0.20, Validation accuracy 94.00
Epoch 6, Train loss 0.23, Train accuracy 93.08, Validation loss 0.18, Validation accuracy 94.46
Epoch 7, Train loss 0.21, Train accuracy 93.53, Validation loss 0.18, Validation accuracy 94.72
Epoch 8, Train loss 0.20, Train accuracy 93.78, Validation loss 0.17, Validation accuracy 95.02
Epoch 9, Train loss 0.19, Train accuracy 94.26, Validation loss 0.16, Validation accuracy 95.15
Epoch 10, Train loss 0.18, Train accuracy 94.54, Validation loss 0.15, Validation accuracy 95.59
Epoch 11, Train loss 0.18, Train accuracy 94.64, Validation loss 0.15, Validation accuracy 95.78
Epoch 12, Train loss 0.17, Train accuracy 94.85, Validation loss 0.14, Validation accuracy 96.01
Epoch 13, Train loss 0.16, Train accuracy 95.00, Validation loss 0.14, Validation accuracy 95.99
Epoch 14, Train loss 0.16, Train accuracy 95.20, Validation loss 0.13, Validation accuracy 96.11
Epoch 15, Train loss 0.15, Train accuracy 95.44, Validation loss 0.13, Validation accuracy 96.17
```



- plot the loss for the training and validation sets using the *plot_loss()* function:

```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()

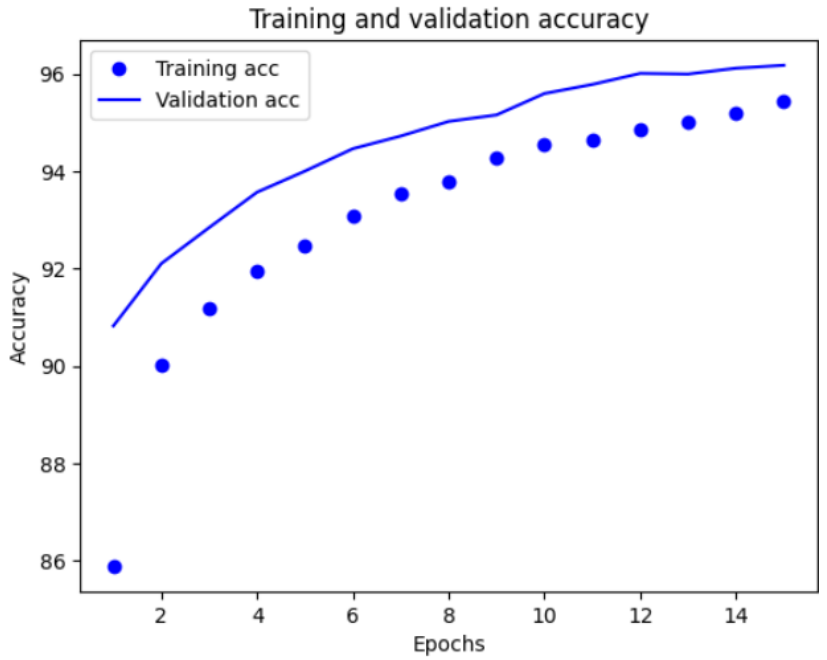
plot_loss(train_loss_all, val_loss_all)
```



- plot the accuracy for the training and validation sets using the *plot_accuracy()* function:

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()

plot_accuracy(train_acc_all, val_acc_all)
```



- evaluate the trained model on the test set, which was not used at all during the training process:

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 0.11, Test accuracy 96.54
```

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

```
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```



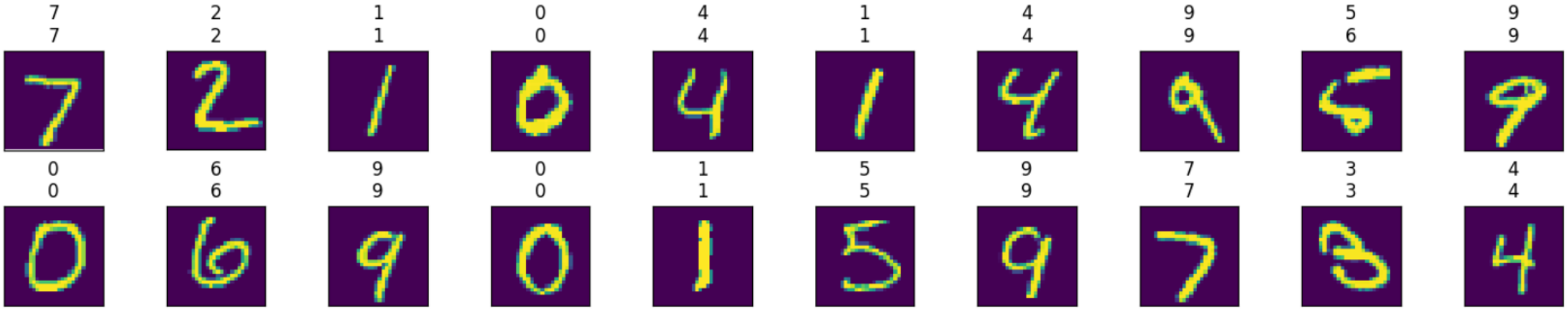
function already done in lab at FashionMNIST dataset section (slide 14)

```python
def get_mnist_labels(labels):
    """Return text labels for the MNIST dataset."""
    text_labels = ['0', '1', '2', '3', '4','5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

adaptation of *get_fashion_mnist_labels()* function from FashionMNIST dataset section (slide 14)

```python
def predict(net, test_iter, n=20):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_mnist_labels(y)
    preds = get_mnist_labels(net(X).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].reshape((n, 28, 28)), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```



New Exercises

Laboratory 8 – Exercises

1. Estimate the following linear model $y = Xw + b + \epsilon$, where $X \in \mathbb{R}^{2000 \times 3}$ consists of 2000 examples, each consisting of 3 features sampled from a standard normal distribution and ϵ obeys a normal distribution with mean 0 and standard deviation of 0.1. The true weights of the model will be $w = [0.15, -0.2, 0.1]^T$ and $b = 0.2$. Create the linear regression model (**do not use PyTorch's predefined layers**) and train it as follows:
   - ➢ set the weights and bias to 3;
   - ➢ set the batch size to 20;
   - ➢ **implement** the mean squared error loss;
   - ➢ **implement** the mini-batch gradient descent optimizer, $lr = 0.02$ (do not use the torch.optim package);
   - ➢ set the total number of epochs to 40.

   After the model is trained, compare the learned model parameters with the actual parameters.

2. Implement the softmax regression model in order to classify the images from the Imagenette dataset (10 classes) as follows:
   - ➢ download the Imagenette train images;
   - ➢ split the train images as follows: validation−4456, training−4456, and the rest of the images represents the testing dataset;
   - ➢ set the batch size to 100;
   - ➢ resize the images to $(300, 100)$ resolution;
   - ➢ initialize the weights at random with zero mean and standard deviation 0.01;
   - ➢ use the cross-entropy loss function;
   - ➢ use mini-batch gradient descent with a learning rate of 0.1 as the optimization algorithm;
   - ➢ set the number of epochs to 5;

   Evaluate the trained model on the test set.

3. Classify the Imagenette dataset (10 classes) using an MLP as follows:
   - ➢ download the Imagenette train images;
   - ➢ split the train images as follows: validation−5000, training−4000, and the rest of the images represents the testing dataset;
   - ➢ set the batch size to 150;
   - ➢ the MLP consists of five hidden layers of 128 units with the rectified linear unit activation function and a dropout layer after the third hidden layer (dropout probability 35%);
   - ➢ set the weights from a normal distribution with $std = 0.01$ and the biases to 0;
   - ➢ set learning rate to 0.15;
   - ➢ train the model for 15 epochs.

   Evaluate the trained model on the test set.

Solutions

1. Estimate the following linear model $y = Xw + b + \epsilon$, where $X \in \mathbb{R}^{2000\times3}$ consists of 2000 examples, each consisting of 3 features sampled from a standard normal distribution and ϵ obeys a normal distribution with mean 0 and standard deviation of 0.1. The true weights of the model will be $w = [0.15, -0.2, 0.1]^T$ and $b = 0.2$. Create the linear regression model (**do not use PyTorch's predefined layers**) and train it as follows:
   - ➢ set the weights and bias to 3;
   - ➢ set the batch size to 20;
   - ➢ **implement** the mean squared error loss;
   - ➢ **implement** the mini-batch gradient descent optimizer, $lr = 0.02$ (do not use the torch.optim package);
   - ➢ set the total number of epochs to 40.

   After the model is trained, compare the learned model parameters with the actual parameters.

```python
import torch
import numpy as np

torch.manual_seed(42)
np.random.seed(42)

def synthetic_data(w, b, num_examples):
    """Generate y = Xw + b + noise."""
    X = torch.normal(0, 1, (num_examples, len(w)))
    y = X @ w + b # Compute the target values y = Xw + b
    y += torch.normal(0, 0.1, y.shape) # Add Gaussian noise with mean 0 and std dev 0.1
    return X, y.reshape(-1, 1)

true_w = torch.tensor([0.15, -0.2, 0.1]) # True weight set to [0.15, -0.2, 0.1]
true_b = 0.2 # True bias set to 0.2
features, labels = synthetic_data(true_w, true_b, 2000) # Generate 2000 data points
```

```python
# Initialize parameters
w = torch.tensor([3.0, 3.0, 3.0], requires_grad=True)  # Initial weights
b = torch.tensor(3.0, requires_grad=True)              # Initial bias

# Define the mean squared error loss function
def loss_fct(y_pred,y):
  return torch.mean((y_pred-y)**2) # Calculate average squared difference

# Parameters for mini-batch gradient descent
lr=0.02
num_epochs = 40
batch_size=20

# Function to load data in mini-batches
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays) # Combine features and labels into a dataset
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train) # DataLoader for batching

# Create data iterator with batch size of 20
data_iter = load_array((features, labels), batch_size)
```

```python
# Training loop with 40 epochs
for epoch in range(num_epochs):
    for X, y in data_iter:
        y_pred = (X @ w + b).reshape(-1, 1) # Compute predictions
        l = loss_fct(y_pred, y) # Compute the loss for the mini-batch
        l.backward() # Compute gradients of the loss with respect to model parameters

        # Manually update parameters without using an optimizer
        with torch.no_grad():
            w.data = w.data - lr * w.grad.data  # Update weights
            b.data = b.data - lr * b.grad.data  # Update bias

        # Zero gradients after each batch to prevent accumulation
        w.grad.data.zero_()
        b.grad.data.zero_()

    # Calculate and print the loss for the entire dataset periodically
    with torch.no_grad():
        y_pred_full = (features @ w + b).reshape(-1, 1)
        total_loss = loss_fct(y_pred_full, labels)
    print(f'Epoch {epoch + 1}, Loss: {total_loss:.6f}')  # Display current epoch and loss
```



```
Epoch 1, Loss: 0.020400
Epoch 2, Loss: 0.009715
Epoch 3, Loss: 0.009733
Epoch 4, Loss: 0.009722
Epoch 5, Loss: 0.009697
Epoch 6, Loss: 0.009714
Epoch 7, Loss: 0.009705
Epoch 8, Loss: 0.009692
Epoch 9, Loss: 0.009704
Epoch 10, Loss: 0.009709
Epoch 11, Loss: 0.009697
Epoch 12, Loss: 0.009728
Epoch 13, Loss: 0.009703
Epoch 14, Loss: 0.009713
Epoch 15, Loss: 0.009702
Epoch 16, Loss: 0.009717
Epoch 17, Loss: 0.009699
Epoch 18, Loss: 0.009719
Epoch 19, Loss: 0.009715
Epoch 20, Loss: 0.009718
Epoch 21, Loss: 0.009708
Epoch 22, Loss: 0.009702
Epoch 23, Loss: 0.009721
Epoch 24, Loss: 0.009709
Epoch 25, Loss: 0.009731
Epoch 26, Loss: 0.009698
Epoch 27, Loss: 0.009691
Epoch 28, Loss: 0.009696
Epoch 29, Loss: 0.009708
Epoch 30, Loss: 0.009727
Epoch 31, Loss: 0.009735
Epoch 32, Loss: 0.009708
Epoch 33, Loss: 0.009713
Epoch 34, Loss: 0.009700
Epoch 35, Loss: 0.009733
Epoch 36, Loss: 0.009727
Epoch 37, Loss: 0.009707
Epoch 38, Loss: 0.009707
Epoch 39, Loss: 0.009713
Epoch 40, Loss: 0.009705
```

```python
# Compare learned parameters with true parameters
print(f"Learned weights: {w.data}")
print(f"True weights: {true_w}")
print(f"Learned bias: {b.data}")
print(f"True bias: {true_b}")
```

```
Learned weights: tensor([ 0.1507, -0.1955,  0.0989])
True weights: tensor([ 0.1500, -0.2000,  0.1000])
Learned bias: 0.19990599155426025
True bias: 0.2
```

```python
from torch import nn   # `nn` is an abbreviation for neural networks

net = nn.Sequential(nn.Linear(3, 1)) # Single-layer linear model with 3 inputs and 1 output

net[0].weight.data.fill_(3) # Set initial weight to 3
net[0].bias.data.fill_(3) # Set initial bias to 3

# Define the mean squared error loss function
def loss_fct(y_pred,y):
  return torch.mean((y_pred-y)**2) # Calculate average squared difference

# Parameters for mini-batch gradient descent
lr=0.02
num_epochs=40
batch_size=20
```

this is the solution using PyTorch's predefined layers

```python
# Function to load data in mini-batches
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays) # Combine features and labels into a dataset
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train) # DataLoader for batching

# Create data iterator with batch size of 20
data_iter = load_array((features, labels), batch_size)
```

```python
# Training loop with 40 epochs
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss_fct(net(X), y) # Compute the loss for the mini-batch
        l.backward() # Compute gradients of the loss with respect to model parameters

        # Manually update parameters without using an optimizer
        net[0].weight.data = net[0].weight.data - lr*net[0].weight.grad.data # Update weight
        net[0].bias.data = net[0].bias.data - lr*net[0].bias.grad.data # Update bias

        # Zero gradients after each batch to prevent accumulation
        net[0].weight.grad.data.zero_()
        net[0].bias.grad.data.zero_()

    # Calculate and print the loss for the entire dataset periodically
    l = loss_fct(net(features), labels)
    print(f'Epoch {epoch + 1}, Loss {l:f}')
```

```python
# Compare learned parameters with true parameters
print(f"Learned weights: {net[0].weight.data}")
print(f"True weights: {true_w}")
print(f"Learned bias: {net[0].bias.data}")
print(f"True bias: {true_b}")
```

```
Learned weights: tensor([[ 0.1503, -0.1985,  0.1000]])
True weights: tensor([ 0.1500, -0.2000,  0.1000])
Learned bias: tensor([0.1978])
True bias: 0.2
```

```
Epoch 1, Loss 0.020631
Epoch 2, Loss 0.009726
Epoch 3, Loss 0.009722
Epoch 4, Loss 0.009697
Epoch 5, Loss 0.009714
Epoch 6, Loss 0.009705
Epoch 7, Loss 0.009692
Epoch 8, Loss 0.009704
Epoch 9, Loss 0.009709
Epoch 10, Loss 0.009697
Epoch 11, Loss 0.009728
Epoch 12, Loss 0.009703
Epoch 13, Loss 0.009713
Epoch 14, Loss 0.009702
Epoch 15, Loss 0.009717
Epoch 16, Loss 0.009699
Epoch 17, Loss 0.009719
Epoch 18, Loss 0.009715
Epoch 19, Loss 0.009718
Epoch 20, Loss 0.009708
Epoch 21, Loss 0.009702
Epoch 22, Loss 0.009721
Epoch 23, Loss 0.009709
Epoch 24, Loss 0.009731
Epoch 25, Loss 0.009698
Epoch 26, Loss 0.009691
Epoch 27, Loss 0.009696
Epoch 28, Loss 0.009708
Epoch 29, Loss 0.009727
Epoch 30, Loss 0.009735
Epoch 31, Loss 0.009708
Epoch 32, Loss 0.009713
Epoch 33, Loss 0.009700
Epoch 34, Loss 0.009733
Epoch 35, Loss 0.009727
Epoch 36, Loss 0.009707
Epoch 37, Loss 0.009707
Epoch 38, Loss 0.009713
Epoch 39, Loss 0.009705
Epoch 40, Loss 0.009701
```

2. Implement the softmax regression model in order to classify the images from the Imagenette dataset (10 classes) as follows:
   - ➢ download the Imagenette train images;
   - ➢ split the train images as follows: validation−4456, training−4456, and the rest of the images represents the testing dataset;
   - ➢ set the batch size to 100;
   - ➢ resize the images to (300, 100) resolution;
   - ➢ initialize the weights at random with zero mean and standard deviation 0.01;
   - ➢ use the cross-entropy loss function;
   - ➢ use mini-batch gradient descent with a learning rate of 0.1 as the optimization algorithm;
   - ➢ set the number of epochs to 5;

   Evaluate the trained model on the test set.

```python
import torchvision
import torch
from torch import nn
from torchvision import transforms
import matplotlib.pyplot as plt
```

- construct the Imagenette dataset and access it in mini-batches:

```python
def load_data_imagenette(batch_size, resize=None):
    """Download the Imagenette dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    imagenette_train = torchvision.datasets.Imagenette(root="../data/train_ex2", split='train', transform=trans, download=True)
    imagenette_train, imagenette_val, imagenette_test = torch.utils.data.random_split(imagenette_train, [4456, 4456, len(imagenette_train)-4456-4456], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(imagenette_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(imagenette_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(imagenette_test, batch_size, shuffle=False, num_workers=2))

# Set the batch size to 100 as specified
batch_size = 100
train_iter, val_iter, test_iter = load_data_imagenette(batch_size, resize=(300, 100))
```

- define the softmax regression model:

```
# Define a simple neural network model with a flatten layer and a linear (fully connected) layer
net = nn.Sequential(nn.Flatten(), nn.Linear(90000, 10)) # 90000 inputs (3 x 300 x 100), 10 outputs (classes)
```

- initialize the parameters:

```
# Initialize the weights of the linear layer as per problem requirements
def init_weights(m):
    if isinstance(m, nn.Linear):  # Check if the layer is nn.Linear
        nn.init.normal_(m.weight, mean=0.0, std=0.01)  # Initialize weights with zero mean and std 0.01

net.apply(init_weights) # Apply the initialization to the model
```

- specify the loss function to be used:

```
# Use cross-entropy loss for multi-class classification
loss = nn.CrossEntropyLoss()
```

- specify the optimizer to be used:

```
# Define a stochastic gradient descent optimizer with learning rate 0.1
lr = 0.1
optimizer = torch.optim.SGD(net.parameters(), lr=lr)
```

- use the *evaluate_accuracy()* function which is already done in lab:

```python
def evaluate_accuracy(net, data_iter):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Simple Example

Suppose a model classifies 100 images, and 90 of those classifications match the true labels. The accuracy would be:

$$\text{Accuracy} = \frac{90}{100} = 0.90 = 90\%$$

- use *train_epoch()* function which is already done in lab:

```python
def train_epoch(net, train_iter, loss, optimizer):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```



train the model for 1 epoch

- use *train()* function which is already done in lab:

```python
def train(net, train_iter, val_iter, loss, num_epochs, optimizer):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')

    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```



train the model for multiple epochs, specified by *num_epochs*

- train the model for 5 epochs by running *train()* function:

```
num_epochs = 5
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, loss, num_epochs, optimizer)
```

```
Epoch 1, Train loss 305.73, Train accuracy 23.50, Validation loss 486.44, Validation accuracy 17.41
Epoch 2, Train loss 273.68, Train accuracy 25.47, Validation loss 304.83, Validation accuracy 19.79
Epoch 3, Train loss 260.24, Train accuracy 26.73, Validation loss 292.38, Validation accuracy 19.41
Epoch 4, Train loss 253.21, Train accuracy 25.34, Validation loss 352.17, Validation accuracy 14.77
Epoch 5, Train loss 250.42, Train accuracy 27.45, Validation loss 223.90, Validation accuracy 21.61
```



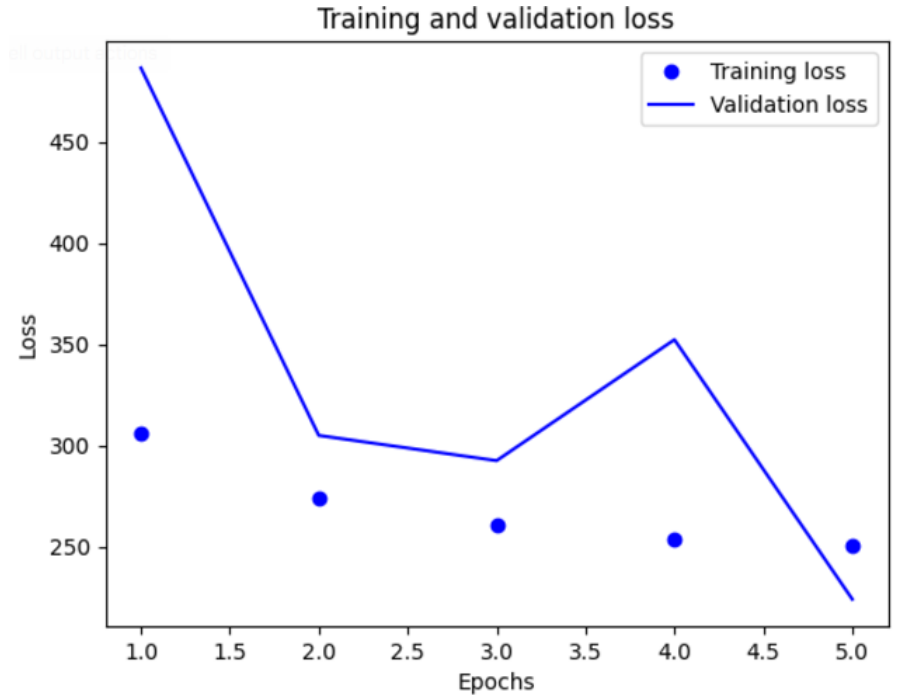
Why such poor performance?

See exercise 3! – slide 101

- plot the loss for the training and validation sets using the *plot_loss()* function:

```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()

plot_loss(train_loss_all, val_loss_all)
```

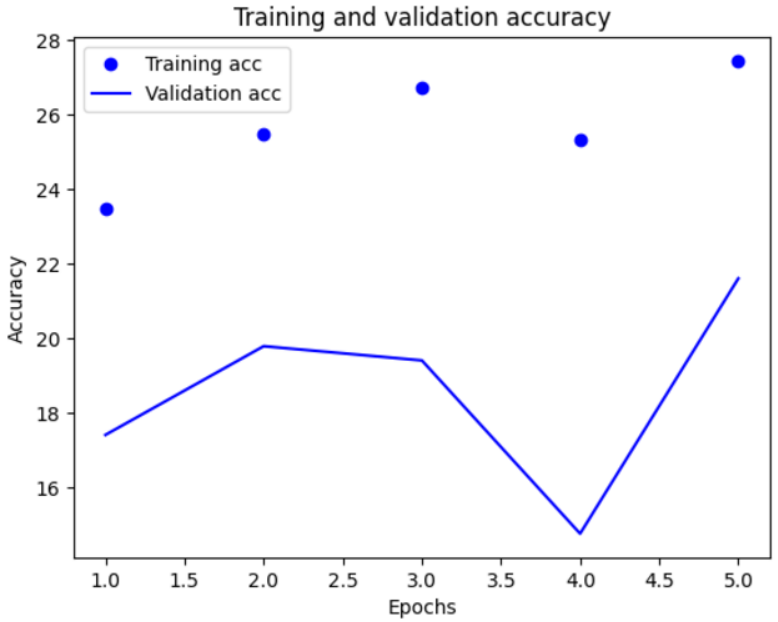




- plot the accuracy for the training and validation sets using the *plot_accuracy()* function:

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()

plot_accuracy(train_acc_all, val_acc_all)
```





- evaluate the trained model on the test set, which was not used at all during the training process:

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 371.95, Test accuracy 17.10
```

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

```
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.permute(1, 2, 0).numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```



function already done in lab at FashionMNIST dataset section (slide 14)

I changed only this because the valid shapes for *ax.imshow()* are:

- grayscale images: *(H,W)*
- RGB images: *(H, W, 3)*

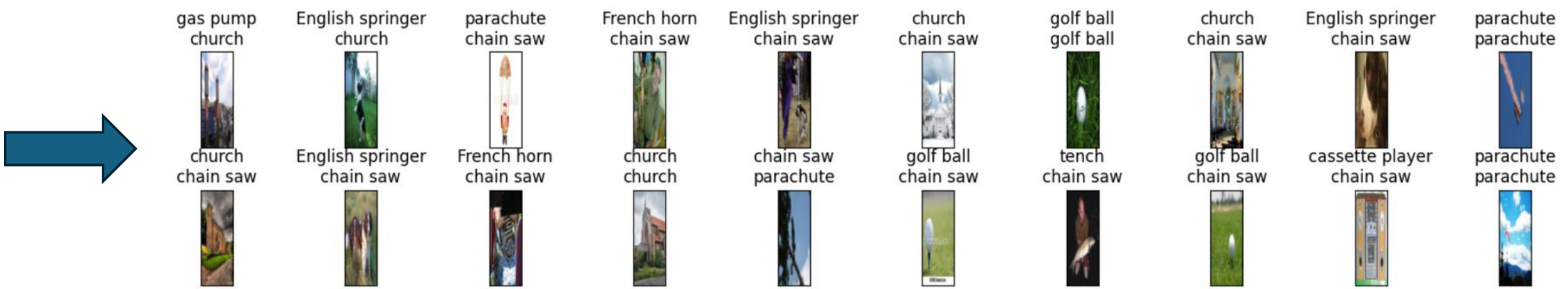
```python
def get_imagenette_labels(labels):
    """Return text labels for the Imagenette dataset."""
    text_labels = ['tench', 'English springer', 'cassette player', 'chain saw',
                   'church','French horn', 'garbage truck', 'gas pump',
                   'golf ball', 'parachute'] # https://github.com/fastai/imagenette#imagenette-1
    return [text_labels[int(i)] for i in labels]
```

adaptation of *get_fashion_mnist_labels()* function from FashionMNIST dataset section (slide 14)

```python
def predict(net, test_iter, n=20):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_imagenette_labels(y)
    preds = get_imagenette_labels(net(X).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].reshape((n, 3, 300, 100)), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```





3. Classify the Imagenette dataset (10 classes) using an MLP as follows:
   ➢ download the Imagenette train images;
   ➢ split the train images as follows: validation−5000, training−4000, and the rest of the images represents the testing dataset;
   ➢ set the batch size to 150;
   ➢ the MLP consists of five hidden layers of 128 units with the rectified linear unit activation function and a dropout layer after the third hidden layer (dropout probability 35%);
   ➢ set the weights from a normal distribution with $std = 0.01$ and the biases to 0;
   ➢ set learning rate to 0.15;
   ➢ train the model for 15 epochs.
   Evaluate the trained model on the test set.

- construct the Imagenette dataset and access it in mini-batches:

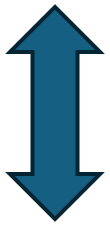
```
def load_data_imagenette(batch_size, resize=None):
    """Download the Imagenette dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    imagenette_train = torchvision.datasets.Imagenette(root="../data/train_ex3", split='train', transform=trans, download=True)
    imagenette_train, imagenette_val, imagenette_test = torch.utils.data.random_split(imagenette_train, [4000, 5000, len(imagenette_train)-4000-5000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(imagenette_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(imagenette_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(imagenette_test, batch_size, shuffle=False, num_workers=2))

# Set the batch size to 150 as specified
batch_size = 150
train_iter, val_iter, test_iter = load_data_imagenette(batch_size, resize=(300, 100))
```

```python
# Define the MLP architecture
net = nn.Sequential(
    nn.Flatten(),  # Flatten the input (e.g., from 3 x 300 x 100 to 90000)
    nn.Linear(90000, 128),  # First hidden layer
    nn.ReLU(),  # Activation function
    nn.Linear(128, 128),  # Second hidden layer
    nn.ReLU(),  # Activation function
    nn.Linear(128, 128),  # Third hidden layer
    nn.ReLU(),  # Activation function
    nn.Dropout(0.35),  # Dropout layer with 35% probability
    nn.Linear(128, 128),  # Fourth hidden layer
    nn.ReLU(),  # Activation function
    nn.Linear(128, 128),  # Fifth hidden layer
    nn.ReLU(),  # Activation function
    nn.Linear(128, 10)  # Output layer for 10 classes
)
```

- the network from exercise 3 gradually reduces the dimensionality of the feature space (e.g., 90000 → 128 → 128 → 128 → 128). This helps the network focus on important features step-by-step and avoid overwhelming the final layer with too many irrelevant details

- the network from exercise 2 directly maps 90,000 inputs to 10 outputs, which can lead to poor weight updates because of noisy gradients and insufficient abstraction



net used here for exercise 3 **vs** net used in exercise 2

```python
# Define a simple neural network model with a flatten layer and a linear (fully connected) layer
net = nn.Sequential(nn.Flatten(), nn.Linear(90000, 10)) # 90000 inputs (3 x 300 x 100), 10 outputs (classes)
```

- the network from exercise 3 includes ReLU activation functions after each layer, introducing non-linearity. This allows the model to capture more complex relationships between features + **dropout** for regularization

- the network from exercise 2 is entirely linear, which limits its ability to model complex data distributions. A purely linear model assumes that the data can be separated by straight hyperplanes, which is unrealistic for image data

the rest is the same as in exercise 2 (excepting learning rate and number of epochs)

```
Epoch 1, Train loss 305.73, Train accuracy 23.50, Validation loss 486.44, Validation accuracy 17.41
Epoch 2, Train loss 273.68, Train accuracy 25.47, Validation loss 304.83, Validation accuracy 19.79
Epoch 3, Train loss 260.24, Train accuracy 26.73, Validation loss 292.38, Validation accuracy 19.41
Epoch 4, Train loss 253.21, Train accuracy 25.34, Validation loss 352.17, Validation accuracy 14.77
Epoch 5, Train loss 250.42, Train accuracy 27.45, Validation loss 223.90, Validation accuracy 21.61
```



```
Epoch 12, Train loss 2.30, Train accuracy 10.42, Validation loss 2.30, Validation accuracy 9.86
Epoch 13, Train loss 2.30, Train accuracy 10.30, Validation loss 2.30, Validation accuracy 9.86
Epoch 14, Train loss 2.30, Train accuracy 10.27, Validation loss 2.30, Validation accuracy 9.70
Epoch 15, Train loss 2.30, Train accuracy 9.98, Validation loss 2.30, Validation accuracy 9.70
```

```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 371.95, Test accuracy 17.10
```



```
test_loss, test_acc = evaluate_accuracy(net, test_iter)
print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')
```

```
Test loss 2.31, Test accuracy 8.10
```

NOTE: training for only **5 epochs** is often insufficient for models to converge, particularly when starting with random initialization and training from scratch on a complex dataset like Imagenette

Thank you !